

# DDR System v6.3

## Reference Manual

This manual is a source-derived reference for DDR System v6.3. All normative DDR facts in this document are derived from the authoritative v6.3 YAML authority pair:

- SEMANTIC AUTHORITY

`ddr_system_v6.3.yaml` - semantic and structural system-definition authority
- MACHINE CONTRACT

`ddr_node_schema_v6.3.yaml` - authority for allowed shapes, conditionals, enums, and validation branching

Interpretive guidance in this manual is limited to explanation and organization. If this manual and the YAML authority ever diverge, the YAML pair controls: the system definition governs semantic content, and the schema governs the allowed validation surface.

### Audit Focus For This Edition

- `document_profile` branching, `project.mode` coupling, and canonical `active_tiers` closure
- lifecycle authority, typed transitions, `prior_status`, and `SUPERSEDE_PENDING` rollback semantics
- canonical operation namespace, `DIRTY` propagation rules, and reconciliation-manifest item structure
- deterministic Express Mode authoring, scan diagnostics, and atomic `UNBUNDLE_EXECUTE`
- extension boundary integrity, ARE activation states, checkpointing, and scoring-profile conformance
- schema conditionals, closed enums, rule-ID typing, and extension-annotation key constraints

### Visual Semantics

| CATEGORY  | SEMANTIC BADGES      | MEANING                           |
|-----------|----------------------|-----------------------------------|
| Authority | <div>NORMATIVE</div> | Identifies whether a statement is |

|                        |                                                                                                                              |                                                                                      |
|------------------------|------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| surfaces               | <div>SCHEMA</div> <div>EXPLANATORY</div> <div>HISTORICAL</div>                                                               | source-controlling, schema-controlling, manual-local explanation, or historical-only |
| Status semantics       | <div>DRAFT</div> <div>ACTIVE</div> <div>DIRTY</div> <div>DEPRECATED</div> <div>SUPERSEDED</div> <div>SUPERSEDE_PENDING</div> | Marks lifecycle condition and operational intent                                     |
| Verification semantics | <div>STRUCTURAL</div> <div>MANUAL</div> <div>SEMANTIC</div>                                                                  | Distinguishes mechanically checkable rules from human-review gates                   |
| Mode and constraints   | <div>FULL</div> <div>EXPRESS</div> <div>LOGICAL</div> <div>PHYSICAL</div>                                                    | Identifies consumption-mode scope and precedence/escalation semantics                |
| Edge semantics         | <div>DERIVES</div> <div>CONSTRAINS</div> <div>IMPLEMENTS</div> <div>EXTENDS</div>                                            | Visually distinguishes the closed relationship vocabulary                            |

**Diagram convention** Each figure visualizes nearby authoritative tables or lists; diagrams aid local comprehension, but the adjacent table or prose remains authoritative.

### Manual Map

- Part I - Authority and Orientation: Sections 1-2 establish source basis, scope, metadata, and high-level change framing.
- Part II - Core Model and Tiers: Sections 3-4 cover the typed DAG model, canonical topology, node contract, and tier-by-tier rule surfaces.
- Part III - Operations, Modes, and Reconciliation: Sections 5-7 cover lifecycle, operations, Express Mode, precedence, reconciliation, and CLEAN-state logic.
- Part IV - Extensions and Validation Surface: Sections 8-9 cover the extension system, ARE, and the schema-side machine contract.
- Part V - Appendices, Quick Reference, and Study Companion: Section 10 provides glossary, history, migration, authoritative counts, source crosswalk material, practitioner quick references, and study aids.

# Table of Contents

1. [Source Basis, Scope, and How to Use This Manual](#)
2. [System Overview and Design Philosophy](#)
3. [Foundational Axioms and Core Structural Model](#)
4. [Tier Reference](#)
5. [Lifecycle and Operations](#)
6. [Consumption Modes and Express Mode](#)
7. [Constraint Precedence, Reconciliation, and CLEAN State](#)
8. [Extension System and ARE](#)
9. [Schema Contract and Machine Validation Surface](#)
  - [9.8 Schema Conditionals, Enums, and Rule-ID Typing](#)
10. [Appendices](#)
  - [10.6 Lifecycle and Operation Quick Reference](#)
  - [10.7 Express Mode Authoring and Unbundling Reference](#)
  - [10.8 ARE and Extension Reference](#)
  - [10.9 Section Summaries](#)
  - [10.10 Pro Tips](#)
  - [10.11 Q&A](#)
  - [10.12 Quiz](#)

# 1. Source Basis, Scope, and How to Use This Manual

This section establishes the authority model for the manual, the scope of the underlying system-definition artifact, the recommended entry points for different readers, and the boundary between normative facts and explanatory organization. Use this section before relying on later sections as reference surfaces.

## 1.1 Authority model

| SOURCE                                 | ROLE                                                                                                |
|----------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>ddr_system_v6.3.yaml</code>      | <b>SEMANTIC AUTHORITY</b> Semantic and structural authority for the DDR System definition           |
| <code>ddr_node_schema_v6.3.yaml</code> | <b>MACHINE CONTRACT</b> Authority for allowed shapes, conditionals, enums, and validation branching |

**Mermaid compatibility policy** This manual uses a `stable-only` Mermaid subset: `flowchart`, `stateDiagram`, `classDiagram`, `sequenceDiagram`, `erDiagram`, `subgraph`, and `classDef`. Renderer-specific v11-only features such as `architecture-beta`, expanded flowchart shapes, and ELK-only layout directives are intentionally excluded from the committed source.

Figure 1.1. Authority precedence and manual role



**Interpretation** The manual depends on both YAML authorities at once: the system definition controls meaning, while the schema controls what valid DDR artifacts may look like.

**Authority basis** Opening authority statement, Section 1.1, and the root roles described by `ddr_system_v6.3.yaml` and `ddr_node_schema_v6.3.yaml`.

The specification defines the current DDR System. The schema defines what valid DDR artifacts may look like. Both are required to document v6.3 correctly.

## 1.2 Scope

| PROPERTY                                              | VALUE                                         |
|-------------------------------------------------------|-----------------------------------------------|
| DDR version                                           | 6.3                                           |
| Document profile of the authoritative source artifact | system_definition                             |
| Project name                                          | DDR System v6.3 - Authoritative Specification |
| Project mode                                          | full                                          |
| System status                                         | Finalized                                     |
| System date                                           | 2026-03-28                                    |
| System scope                                          | Systems-, language-, and domain-agnostic      |
| Authority                                             | DDR Architecture Board                        |
| Lineage                                               | Supersedes DDR v6.2                           |

### 1.3 How to use this manual

| ROLE                    | PRIMARY NEED                                                                                                  | START HERE                               |
|-------------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------|
| New reader              | Understand the authority model, current-state overview, and the core DDR shape before diving into details     | Sections 1, 2, then 3                    |
| Tier author / reviewer  | Check what each tier must contain and how it interacts with parent, child, and reconciliation rules           | Sections 4, 3, and 7                     |
| Validator / tool author | Inspect machine-facing profile rules, schema branches, lifecycle transitions, guards, and validation surfaces | Sections 9, 5, and 3                     |
| Extension implementer   | Confirm extension boundaries, ARE behavior, schema-side extension rules, and reconciliation touchpoints       | Sections 8, 9, and 7                     |
| Audit / history reader  | Trace changes from current-state metadata through version history, migration, counts, and source crosswalk    | Sections 2.3, 10.2, 10.3, 10.4, and 10.5 |

**Figure 1.2. Reader routing by technical objective**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: Reader routing by technical objective accDescr: Maps common reader roles to the sections that best answer their immediate technical question.
subgraph ROLES["Reader roles"]
NR["New reader"]
TA["Tier author or reviewer"]
VT["Validator or tool author"]
EI["Extension implementer"]
AH["Audit or history reader"]
end
subgraph PATHS["Recommended entry points"]
O12["Sections 1-2 authority and overview"]
C3["Section 3 core structural model"]
T4["Section 4 tier reference"]
R7["Section 7 reconciliation and CLEAN"]
S9["Section 9 schema contract"]
L5["Section 5 lifecycle and operations"]
E8["Section 8 extension system and ARE"]
CH["Sections 2.3 and 10.2-10.5 history, migration, counts, and crosswalk"]
end
NR --> O12
NR --> C3
TA --> T4
TA --> R7
VT --> S9
VT --> L5
EI --> E8
EI --> S9
AH --> CH
classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px;
classDef schema fill:rgba(59,130,246,0.1),stroke:rgba(59,130,246,0.5),color:#60a5fa,stroke-width:2px;
class NR,TA,VT,EI,AH schema;
class O12,C3,T4,R7,S9,L5,E8,CH normative;
```

**Interpretation** The routing matrix is optimized for lookup speed: readers enter through the authority, structure, operational, extension, or history surfaces that match their immediate task.

**Authority basis** Section 1.3 role-routing table and the preserved section-family structure of this manual.

## 1.4 Normative vs explanatory content

- **NORMATIVE DDR FACTS** are taken directly from the authoritative YAML files.
- **REFERENCE EXPLANATIONS** reorganize or summarize those facts for lookup.
- **SCHEMA-LITERAL CONSTRAINTS** remain governed by the schema whenever a rule depends on field shape, enum closure, or conditional validation branching.
- **Examples** in this manual are limited to source-native examples already present in the authoritative files, such as representative nodes, canonical tier variants, scoring profiles, lifecycle transitions, and extension catalog entries.

## 2. System Overview and Design Philosophy

This section covers the current authoritative metadata, the governing design philosophy, the explicit v6.2-to-v6.3 change surface, and the current errata state. Use Section 3 for the operational structure of the Core model and Section 10 for historical and migration context.

### 2.1 System metadata

| SURFACE                                             | VALUE                                                                                                                                                                                  |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>project.name</code>                           | DDR System v6.3 - Authoritative Specification                                                                                                                                          |
| <code>project.created</code>                        | 2026-02-26                                                                                                                                                                             |
| <code>project.mode</code>                           | full                                                                                                                                                                                   |
| <code>system_metadata.status</code>                 | Finalized                                                                                                                                                                              |
| <code>system_metadata.date</code>                   | 2026-03-28                                                                                                                                                                             |
| <code>system_metadata.scope</code>                  | Systems-, language-, and domain-agnostic                                                                                                                                               |
| <code>system_metadata.authority</code>              | DDR Architecture Board                                                                                                                                                                 |
| <code>system_metadata.lineage</code>                | Supersedes DDR v6.2                                                                                                                                                                    |
| <code>system_metadata.single_source_of_truth</code> | This document is the exclusive normative specification for DDR v6.3; prior versions, conversation records, partial specifications, and derivative documents carry no normative weight. |

The authoritative specification does more than identify itself as current. It explicitly closes the authority chain: the YAML system-definition artifact is the normative source of truth, and derivative documents such as this manual are explanatory aids only.

### 2.2 Design philosophy

| PRINCIPLE       | DESCRIPTION                                                                                                                               |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Minimize Design | Every element earns its existence. No tier, edge type, operation, or rule exists without a concrete problem it solves. The system must be |

|                               |                                                                                                                                                                                                                              |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Complexity                    | adoptable by a solo developer on day one and scale to enterprise without structural changes.                                                                                                                                 |
| Avoid Premature Optimization  | The Core defines the minimum viable graph. Advanced analytical capabilities, inference engines, and domain-specific intelligence are delivered exclusively via optional Extensions. The Core never anticipates an Extension. |
| Maximize Structural Integrity | The DAG is the single source of truth. Every node is traceable, every edge is typed, every mutation is validated. The system is correct by construction.                                                                     |

## 2.3 Changes from v6.2 in v6.3

| AREA                               | PRIOR                                                                                                                                             | CURRENT                                                                                                                                                                 | RATIONALE                                                                                                                                       |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Explicit document profiling        | System-definition intent was inferred indirectly from <code>system_metadata</code>                                                                | <code>document_profile</code> explicitly distinguishes <code>project_instance</code> , <code>project_instance_express</code> , and <code>system_definition</code> roots | Makes authored document intent machine-explicit and lets the schema require the full authoritative surface only for system-definition artifacts |
| Topology closure                   | <code>active_tiers</code> enforced membership loosely and left topology consequences to downstream logic                                          | <code>active_tiers</code> is restricted to canonical order variants and topology obligations become first-class invariants                                              | Closes ordering and representative-coverage ambiguity without adding a second topology model                                                    |
| Lifecycle authority simplification | Lifecycle behavior was split across <code>status_transitions</code> and <code>prohibited_transitions</code> , with composite raw operation tokens | <code>status_transitions</code> is the sole lifecycle authority and transition metadata decomposes operation, phase, side-effect, and prerequisites                     | Removes dual-authority drift and keeps operation identity machine-normalized                                                                    |
| ARE contract hardening             | ARE activation states, E5 <code>scoring_profile</code> requirements, and custom-profile                                                           | Activation states are structurally typed, E5 requires <code>scoring_profile</code> ,                                                                                    | Strengthens the schema front door while leaving profile resolution                                                                              |



|                                   |                                                                                                    |                                                                                                                                   |                                                                                                           |
|-----------------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
|                                   | structure were under-typed                                                                         | and custom-profile shape is explicit                                                                                              | and score-band ordering to deterministic ARE conformance validation                                       |
| Operation namespace normalization | Operations, lifecycle rows, and scaffold comments mixed canonical names with phase or alias tokens | The canonical operation surface is closed and <code>UNBUNDLE_EXECUTE</code> is the sole commit-phase token                        | Keeps validators, logs, and API-like surfaces aligned                                                     |
| Rule identifier typing            | Invariant, atomic-rule, and extension-rule identifiers were only partially typed                   | Rule-ID families are centralized and typed consistently across the schema                                                         | Reduces malformed-reference drift                                                                         |
| Express mode closure              | Express Mode groups and top-level express authority were under-enforced                            | Group compositions are fixed structurally and express-capable profiles require the full <code>express_mode</code> authority block | Prevents authored Express Mode files from redefining group structure or omitting their governing contract |

## 2.4 Errata state

The authoritative `errata_log` is empty. No active errata entries are carried in the v6.3 system-definition artifact.

**Figure 2.1. v6.3 framing surfaces and downstream impact**

```

%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor':
'#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a',
'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize':
'14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: v6.3 framing surfaces and downstream impact accDescr: Connects system metadata, the
v6.3 change surface, and the empty errata log to the later sections they frame. subgraph
BASE["Authoritative framing surfaces"] META["system metadata"] ERR["errata_log is empty"] end
subgraph CHANGE["v6.3 change surface"] PROF["document_profile and active_tiers closure"]
LIFE["lifecycle and operation normalization"] EXP["Express and ARE hardening"] end subgraph
DOWN["Downstream reading surfaces"] CORE["Sections 3-4 core model and tiers"] OPS["Sections
5-7 operations, modes, and CLEAN"] AUD["Section 10.4 counts and errata state"] end META -->
CORE META --> OPS PROF --> CORE LIFE --> OPS EXP --> OPS ERR --> AUD classDef normative
fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px; classDef schema
fill:rgba(59,130,246,0.1),stroke:rgba(59,130,246,0.5),color:#60a5fa,stroke-width:2px; class
META,ERR,CORE,OPS,AUD normative; class PROF,LIFE,EXP schema;

```

**Interpretation** Section 2 is not introductory filler; it frames the exact v6.3 deltas that explain why later profile, lifecycle, Express, and ARE sections are structured as they are.

**Authority basis** `project`, `system_metadata`, `errata_log`, and the Section 2.3 change table sourced from `ddr_system_v6.3.yaml`.

# 3. Foundational Axioms and Core Structural Model

This section covers the current-state structural foundation of DDR v6.3: axioms, document profiles, canonical topology, universal node shape, edge semantics, citation rules, and invariants. Use Section 4 for tier-local rule surfaces, Section 5 for operational behavior, and Section 9 for the corresponding schema-side machine contract.

## 3.1 Foundational axioms

| ID   | NAME                  | STATEMENT                                                                                                                 | IMPLICATION                                                                                                         |
|------|-----------------------|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| AX-1 | Traceability          | Every non-root node must cite at least one parent via a typed edge.                                                       | Complete audit trails from intent to implementation; no orphaned requirements.                                      |
| AX-2 | Abstraction Ordering  | Technology and implementation specificity are deferred until logically necessary.                                         | Tiers above CL (XPD, SIL, GPCL, FCL) must contain no technology, hardware, or implementation references.            |
| AX-3 | Determinism           | Identical inputs produce unambiguous, mechanically verifiable outputs.                                                    | Structural rules support automated validation; semantic rules require explicit human disposition before activation. |
| AX-4 | Universality          | The Core applies to all software systems regardless of domain, scale, or technology.                                      | No domain-specific assumptions belong in any Core tier.                                                             |
| AX-5 | Extensibility         | Advanced analytical capabilities are delivered exclusively via optional Extensions.                                       | Core structure remains stable and does not depend on Extension behavior.                                            |
| AX-6 | Declarative Integrity | The Core is strictly declarative; all inference, optimization, and automated recommendation are Extension-only behaviors. | Core structural invariants cannot be destabilized by analytical logic.                                              |
| AX-7 | DAG Acyclicity        | No citation chain may produce a cycle; causality flows in one direction only.                                             | Graph traversal always terminates.                                                                                  |

3.2 Document profiles

The schema defines three top-level document profiles:

| PROFILE                  | MEANING                                  | HIGH-LEVEL CONTRACT                                                                                                                                                                            |
|--------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| project_instance         | Lean full-mode project artifact          | Rooted in ddr_version, document_profile, active_tiers, and nodes; must not require system_metadata                                                                                             |
| project_instance_express | Express-mode project artifact            | Same lean root plus express obligations; requires express_mode, and each node must carry express_mode_group                                                                                    |
| system_definition        | Authoritative DDR specification artifact | Requires the full normative top-level surface, including metadata, axioms, edge definitions, tier definitions, operations, extension system, compliance, glossary, ARE profiles, and lifecycle |

Additional machine-enforced coupling rules that matter during authoring:

- If `project.mode = express`, the schema forces `document_profile = project_instance_express`.
- If `document_profile = project_instance_express` and `project.mode` is present, `project.mode` must equal `express`.
- If `document_profile = project_instance_express`, every node must carry `express_mode_group`.
- If `active_tiers` contains `XPD`, `SIL` is no longer schema-legal as a root node and must carry at least one `parent_ids` entry.

Authoring examples:

| INTENT                          | MINIMAL VALID PROFILE POSTURE                                                                                                                                                                             |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lean full-mode project file     | <code>document_profile: project_instance</code> plus canonical <code>active_tiers</code> and <code>nodes</code>                                                                                           |
| Lean express-mode project file  | <code>document_profile: project_instance_express</code> , top-level <code>express_mode</code> , node-level <code>express_mode_group</code> , and <code>project.mode: express</code> if project is present |
| Authoritative DDR spec artifact | <code>document_profile: system_definition</code> plus the full normative surface required by the schema                                                                                                   |

**Figure 3.1.** `document_profile` branching and required-surface split

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: document_profile branching and required-surface split accDescr: Shows how the top-level schema root branches into three document profiles and which surface each branch must carry.
ROOT["Top-level schema root"] subgraph PROFILES["document_profile branches"]
PI["project_instance"] PIE["project_instance_express"] SD["system_definition"] end subgraph
SURFACES["Required surface"] LEAN["Lean root: ddr_version, document_profile, active_tiers, nodes"] EXPRESS["Lean root plus express_mode and node express_mode_group"] FULL["Lean root plus metadata, axioms, operations, extensions, compliance, glossary, and lifecycle"] end
ROOT --> LEAN ROOT --> PIE --> EXPRESS ROOT --> SD --> FULL
classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px;
classDef schema fill:rgba(59,130,246,0.1),stroke:rgba(59,130,246,0.5),color:#60a5fa,stroke-width:2px;
class ROOT,PI,PIE,SD schema; class LEAN,EXPRESS,FULL normative;
```

**Interpretation** `document_profile` is the top-level branching key that separates lean project artifacts from the authoritative full-surface system-definition artifact.

**Authority basis** `ddr_node_schema_v6.3.yaml` top-level profile branching and Section 3.2.

**3.3 Canonical `active_tiers` variants**

The schema allows exactly four ordered variants:

| VARIANT              | ORDERED TIERS                               |
|----------------------|---------------------------------------------|
| Base 7-tier topology | SIL, GPCL, FCL, SAL, ICL, CDL, ISL          |
| Base + XPD           | XPD, SIL, GPCL, FCL, SAL, ICL, CDL, ISL     |
| Base + CL            | SIL, GPCL, FCL, CL, SAL, ICL, CDL, ISL      |
| Base + XPD + CL      | XPD, SIL, GPCL, FCL, CL, SAL, ICL, CDL, ISL |

**Figure 3.2. Canonical active\_tiers closure**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart TB
accTitle: Canonical active_tiers closure accDescr: Lists the only four ordered tier sets permitted by the schema for active_tiers. SCHEMA["active_tiers oneOf closure"] subgraph CL_OFF["CL inactive variants"] V1["SIL -> GPCL -> FCL -> SAL -> ICL -> CDL -> ISL"] V2["XPD -> SIL -> GPCL -> FCL -> SAL -> ICL -> CDL -> ISL"] end subgraph CL_ON["CL active variants"] V3["SIL -> GPCL -> FCL -> CL -> SAL -> ICL -> CDL -> ISL"] V4["XPD -> SIL -> GPCL -> FCL -> CL -> SAL -> ICL -> CDL -> ISL"] end
SCHEMA --> V1 SCHEMA --> V2 SCHEMA --> V3 SCHEMA --> V4 classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px; classDef schema fill:rgba(59,130,246,0.1),stroke:rgba(59,130,246,0.5),color:#60a5fa,stroke-width:2px; class SCHEMA schema; class V1,V2,V3,V4 normative;
```

**Interpretation** DDR v6.3 closes `active_tiers` to four legal ordered topologies; arbitrary activation sets and arbitrary orderings are not part of the allowed model.

**Authority basis** `ddr_node_schema_v6.3.yaml` `properties.active_tiers` closure and Section 3.3.

3.4 Canonical topology and representative nodes

**Figure 3.3. Canonical topology and merge behavior**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: Canonical topology and merge behavior accDescr: Shows the representative DDR tier topology, including the CL constraint branch into SAL and the implements chain below ICL
subgraph INTENT["Intent and governance spine"] XPD["XPD"] SIL["SIL"] GPCL["GPCL"] FCL["FCL"] end
subgraph BOUNDS["Constraint branch"] CL["CL declared constraints"] end
subgraph DELIVERY["Architecture and realization path"] SAL["SAL"] ICL["ICL"] CDL["CDL"] ISL["ISL"] end
XPD -->|derives| SIL
SIL -->|derives| GPCL
GPCL -->|derives| FCL
FCL -->|derives| SAL
SAL -->|derives| CL
CL -->|constrains| SAL
SAL -->|derives| ICL
ICL -->|implements| CDL
CDL -->|implements| ISL
classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px;
classDef caution fill:rgba(245,158,11,0.1),stroke:rgba(245,158,11,0.5),color:#fbbf24,stroke-width:2px;
class XPD,SIL,GPCL,FCL,SAL,ICL,CDL,ISL normative; class CL caution;
```

**Interpretation** `SAL` is the architectural merge point: capability intent arrives from `FCL`, while declared technology and infrastructure bounds arrive from `CL` through `constrains`.

**Authority basis** Representative nodes, tier\_definitions, and the CIT-R3 / INV-4 surfaces summarized in Sections 3.4, 3.8, and 3.9.

The authoritative system-definition artifact includes one representative node for each active tier:

| NODE | TIER | TITLE | PARENT CITATIONS |
|------|------|-------|------------------|
|------|------|-------|------------------|

| ID       |      |                                    |                                                          |
|----------|------|------------------------------------|----------------------------------------------------------|
| XPD-0.1  | XPD  | Existential Purpose Document       | root                                                     |
| SIL-1.1  | SIL  | Strategic Intent Layer             | XPD-0.1 <b>via</b> derives <b>with</b> semantic          |
| GPCL-2.1 | GPCL | Governance, Policy & Quality Layer | SIL-1.1 <b>via</b> derives <b>with</b> traceability      |
| FCL-3.1  | FCL  | Functional Capability Layer        | GPCL-2.1 <b>via</b> derives <b>with</b> semantic         |
| CL-4.1   | CL   | Constraint Layer                   | FCL-3.1 <b>via</b> derives <b>with</b> semantic          |
| SAL-5.1  | SAL  | System Architecture Layer          | FCL-3.1 <b>via</b> derives; CL-4.1 <b>via</b> constrains |
| ICL-6.1  | ICL  | Interface & Contracts Layer        | SAL-5.1 <b>via</b> derives <b>with</b> semantic          |
| CDL-7.1  | CDL  | Component Design Layer             | ICL-6.1 <b>via</b> implements                            |
| ISL-8.1  | ISL  | Implementation Scaffold Layer      | CDL-7.1 <b>via</b> implements                            |

### 3.5 Universal node format

The specification documents 13 node schema fields:

| FIELD             | TYPE                            | NOTES                                                                                                                |
|-------------------|---------------------------------|----------------------------------------------------------------------------------------------------------------------|
| id                | TIER-N.M                        | Immutable once assigned                                                                                              |
| tier              | enum                            | One of XPD, SIL, GPCL, FCL, CL, SAL, ICL, CDL, ISL                                                                   |
| title             | string                          | Human-readable artifact label                                                                                        |
| content           | text                            | Body constrained by the tier's atomic ruleset                                                                        |
| parent_ids        | list of ParentCitation          | Required for all non-root nodes; SIL is root unless XPD is active. Legal edge types: derives, constrains, implements |
| status            | enum                            | DRAFT, ACTIVE, DIRTY, DEPRECATED, SUPERSEDED, SUPERSEDE_PENDING                                                      |
| constraint_origin | enum, conditional               | CL only; one of derived, imposed                                                                                     |
| prior_status      | status enum subset, conditional | Only for SUPERSEDE_PENDING; allowed values are ACTIVE, DEPRECATED, DIRTY                                             |

|                       |                   |                                                                                                                                                       |
|-----------------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| version               | SemVer string     | Incremented on <code>MODIFY</code>                                                                                                                    |
| created               | ISO 8601 datetime | Creation timestamp                                                                                                                                    |
| modified              | ISO 8601 datetime | Last modification timestamp                                                                                                                           |
| express_mode_group    | enum, conditional | Required when <code>document_profile = project_instance_express</code> ; one of <code>G1</code> , <code>G2</code> , <code>G3</code> , <code>G4</code> |
| extension_annotations | map               | Read-only extension metadata with reserved shadow-key blocking                                                                                        |

**Figure 3.4.** `DdrNode` and `ParentCitation` conditional structure

```

%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor':
'#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a',
'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize':
'14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% classDiagram
accTitle: DdrNode and ParentCitation conditional structure accDescr: Shows the closed node
contract, the typed parent citation surface, and the conditional branches that activate express-
mode, CL-only, and SUPERSEDE_PENDING-only fields. class DdrNode { +id: NodeId +tier: TierEnum
+title: string +content?: text +parent_ids: ParentCitation[] +status: StatusEnum
+constraint_origin?: derived|imposed +prior_status?: ACTIVE|DEPRECATED|DIRTY +version: semver
+created: date-time +modified: date-time +express_mode_group?: G1|G2|G3|G4
+extension_annotations?: namespaced map } class ParentCitation { +id: NodeId +edge_type:
derives|constrains|implements +derivation_mode?: semantic|traceability } class
ExpressProfileCondition { +applies_when: document_profile = project_instance_express
+requires: express_mode_group } class CCondition { +applies_when: tier = CL +allows:
constraint_origin } class SupersedePendingCondition { +applies_when: status =
SUPERSEDE_PENDING +requires: prior_status } DdrNode "0..*" --> "1" ParentCitation : parent_ids
DdrNode ..> ExpressProfileCondition : conditional DdrNode ..> CCondition : conditional
DdrNode ..> SupersedePendingCondition : conditional

```

**Interpretation** The node contract is closed, but several fields are legal only under profile-, tier-, or status-specific branches that the schema enforces explicitly.

**Authority basis** `ddr_node_schema_v6.3.yaml` `$defs.DdrNode`, `$defs.ParentCitation`, and Section 3.5.

### 3.6 Node ID format

| ITEM            | VALUE                                |
|-----------------|--------------------------------------|
| General pattern | <code>[TIER]-[SECTION].[ITEM]</code> |
|                 |                                      |



|                   |                                                                                        |
|-------------------|----------------------------------------------------------------------------------------|
| XPD pattern       | XPD-0.N                                                                                |
| Examples          | SIL-1.3, GPCL-2.1, CDL-12.5, XPD-0.1                                                   |
| Immutability rule | IDs never change. A superseded node retains its ID; the replacement receives a new ID. |

**Figure 3.5. Node ID grammar decomposition**

```

%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor':
'#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a',
'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize':
'14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: Node ID grammar decomposition accDescr: Breaks a DDR node ID into its tier token,
punctuation tokens, section ordinal, item ordinal, and the XPD root exception. ID["Node ID"]
subgraph GENERAL["General form"] TIER["Tier token: XPD|SIL|GPCL|FCL|CL|SAL|ICL|CDL|ISL"] DASH["-"]
SECTION["Section ordinal"] DOT["."] ITEM["Item ordinal"] end ROOT["Special root branch: XPD-
0.N"] ID --> TIER TIER --> DASH --> SECTION --> DOT --> ITEM TIER -. XPD only .-> ROOT classDef
normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px;
classDef schema fill:rgba(59,130,246,0.1),stroke:rgba(59,130,246,0.5),color:#60a5fa,stroke-
width:2px; class ID,TIER,SECTION,ITEM,ROOT normative; class DASH,DOT schema;

```

**Interpretation** The ID format is stable across the system, with `XPD-0.N` as the only special root-pattern branch.

**Authority basis** `node_id_format` in `ddr_system_v6.3.yaml` and the Section 3.6 lookup table.

### 3.7 Edge types

The current edge vocabulary is the four-type surface declared by `edge_type_definitions`. The specification also records the consolidation decision that reduced older edge vocabularies into this set.

| EDGE TYPE         | SYMBOL          | SEMANTICS                                                                                                                                                                                                                               |
|-------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>DERIVES</b>    | --derives-->    | Child content derives from parent requirements, or parent is cited as authoritative lineage.<br>Optional <code>derivation_mode</code> may be <code>semantic</code> or <code>traceability</code> ; omitted means <code>semantic</code> . |
| <b>CONSTRAINS</b> | --constrains--> | Parent sets enforceable limits on the child's design space.                                                                                                                                                                             |
| <b>IMPLEMENTS</b> | --implements--> | Child provides concrete realization of the parent's abstract specification.                                                                                                                                                             |
| <b>EXTENDS</b>    | ...extends...>  | Extension reads or annotates Core nodes without modifying Core semantics.                                                                                                                                                               |

### 3.8 Citation rules

| RULE   | STATEMENT                                                                                                                                                                                                                              |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CIT-R1 | Every non-root node must have $\geq 1$ parent citation; only root nodes may have an empty <code>parent_ids</code> array.                                                                                                               |
| CIT-R2 | Parent citations must reference the immediately preceding active tier or tiers. For <code>derives</code> , <code>derivation_mode</code> may be <code>semantic</code> or <code>traceability</code> ; default is <code>semantic</code> . |
| CIT-R3 | <code>CL -&gt; SAL</code> constraint edges are stored in <code>parent_ids</code> with edge type <code>constrains</code> .                                                                                                              |
| CIT-R4 | Any inline <code>[TIER-N.M]</code> citation in node content must have a matching <code>parent_ids</code> entry.                                                                                                                        |
| CIT-R5 | Extension <code>extends</code> relationships are stored in <code>extension_annotations</code> only, never in <code>parent_ids</code> .                                                                                                 |
| CIT-R6 | Any authority-linkage <code>derives</code> edge must set <code>derivation_mode: traceability</code> ; non- <code>derives</code> edges must not carry <code>derivation_mode</code> .                                                    |
| CIT-R7 | A child may remain <code>ACTIVE</code> only while each cited parent remains at the version last validated against. Any parent <code>MODIFY</code> or <code>SUPERSEDE</code> that changes cited content forces child re-validation.     |

Figure 3.6. Edge vocabulary and citation-surface separation

```

%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor':
'#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a',
'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize':
'14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: Edge vocabulary and citation-surface separation accDescr: Distinguishes the three legal
Core parent citation edge types from the extension-only extends relationship. subgraph
CORE["Core citation surface"] PARENT["parent_ids[]"] DER["derives"] CON["constrains"]
IMP["implements"] MODE["derivation_mode: semantic|traceability"] end subgraph
EXTENSION["Extension-only surface"] EXTANN["extension_annotations"] EXTEDGE["extends"] end
FORBID["ParentCitation.edge_type excludes extends"] PARENT --> DER PARENT --> CON PARENT --
> IMP DER --> MODE EXTANN --> EXTEDGE EXTEDGE -. extension overlay only .-> FORBID FORBID
-. never stored in .-> PARENT classDef normative
fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px; classDef schema
fill:rgba(59,130,246,0.1),stroke:rgba(59,130,246,0.5),color:#60a5fa,stroke-width:2px; classDef alert
fill:#ffebee,stroke:#c62828,color:#b71c1c,stroke-width:1.5px; class PARENT,EXTANN,MODE
normative; class DER,CON,IMP,EXTEDGE schema; class FORBID alert;

```

**Interpretation** Core parent citations are closed to three edge types, while `extends` is isolated into extension annotations so that analytical overlays cannot masquerade as Core lineage.

**Authority basis** `edge_type_definitions`, `citation_rules`,  
`$defs.ParentCitation`, and Sections 3.7–3.8.

Worked citation examples:

```
# Semantic derivation: the child's design content is derived from the parent's requirements.
parent_ids:
  - id: GPCL-2.1
    edge_type: derives
    derivation_mode: semantic

# Traceability citation: the parent is cited as lineage authority rather than as source semantics.
parent_ids:
  - id: SIL-1.1
    edge_type: derives
    derivation_mode: traceability

# Constraint injection into SAL.
parent_ids:
  - id: CL-4.1
    edge_type: constrains

# Implementation realization.
parent_ids:
  - id: ICL-6.1
    edge_type: implements
```

Common failure cases the YAML pair rejects or treats as non-conformant:

- `derivation_mode` attached to `constrains` or `implements`
- `extends` authored inside `parent_ids`
- inline `[TIER-N.M]` citations that do not have a matching `parent_ids` entry
- children left `ACTIVE` after cited-parent content changes without re-validation under `CIT-R7`

3.9 DAG invariants

| INVARIANT | STATEMENT                                                                                                                                                                                                                                                  |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| INV-1     | No cycles are permitted at any path length.                                                                                                                                                                                                                |
| INV-2     | No tier-skipping: citations must target the immediately preceding active tier or tiers. <code>SAL</code> is the only exhaustive merge-node exception.                                                                                                      |
| INV-3     | <code>active_tiers</code> must be one of the four canonical ordered sets. Every node tier must belong to <code>active_tiers</code> , and every <code>system_definition</code> artifact must include at least one representative node for each active tier. |
| INV-4     | When <code>CL</code> is inactive, <code>SAL</code> derives directly from <code>FCL</code> .                                                                                                                                                                |
| INV-5     | All non-root nodes must carry at least one parent citation.                                                                                                                                                                                                |
| INV-6     | <code>SUPERSEDE</code> must be atomic across all tiers. Partial application is a structural violation. At most one <code>XPD</code> node may carry status <code>ACTIVE</code> at                                                                           |

|       |                                                                                                                                                                                                                                         |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       | any time.                                                                                                                                                                                                                               |
| INV-7 | Structural validity may coexist with declared semantic gaps only when the gap is explicitly logged in the reconciliation manifest under an allowed classification, with human rationale and required resolution or waiver before CLEAN. |
| INV-8 | <code>lifecycle.status_transitions</code> must form a complete and closed state machine: every non-terminal status has at least one valid outbound transition, and undefined transitions are invalid.                                   |

See also: Section 4, Section 5, Section 9.

## 4. Tier Reference

This section covers the tier-by-tier current-state contract for DDR v6.3, including representative nodes, parent and child relationships, inclusion rules, exclusion rules, and tier-specific verification notes. Use Section 3 for shared structural rules, Section 5 for lifecycle and operation effects, and Section 7 for reconciliation and CLEAN-state implications.

**Figure 4.1. Abstraction descent across active tiers**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: Abstraction descent across active tiers accDescr: Groups the DDR tiers into purpose, capability, and realization bands to show the system's downward abstraction path.
subgraph WHY["Purpose and direction"]
XPD["XPD ethical purpose"]
SIL["SIL strategic intent"]
GPCL["GPCL governance and quality"]
end
subgraph WHAT["Capability and bounds"]
FCL["FCL functional capability"]
CL["CL declared technology bounds"]
end
subgraph HOW["Architecture and realization"]
SAL["SAL architecture"]
ICL["ICL contracts and schemas"]
CDL["CDL component design"]
ISL["ISL implementation scaffolding"]
end
XPD --> SIL --> GPCL --> FCL --> SAL --> ICL --> CDL --> ISL
FCL --> CL
classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px;
classDef caution fill:rgba(245,158,11,0.1),stroke:rgba(245,158,11,0.5),color:#fbbf24,stroke-width:2px;
class XPD,SIL,GPCL,FCL,SAL,ICL,CDL,ISL normative;
class CL caution;
```

**Interpretation** DDR descends from ethical and strategic abstraction into concrete contracts and scaffolding, while **CL** sits beside the functional path as a declaration of non-negotiable bounds rather than user-facing behavior.

**Authority basis** tier\_definitions , AX-2 , and the tier reference surfaces in Section 4 .

**Figure 4.2. Constraint injection path from CL into downstream design**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: Constraint injection path from CL into downstream design accDescr: Shows CL as a declared constraint surface that bounds SAL and therefore all downstream contracts, design, and scaffolding.
subgraph INTENT["Capability intent"]
FCL["FCL capability intent"]
end
subgraph CONSTRAINTS["Declared bounds"]
CL["CL declared constraints"]
end
subgraph DESIGN["Downstream realization"]
SAL["SAL architecture synthesis"]
ICL["ICL interfaces and schemas"]
CDL["CDL component blueprints"]
ISL["ISL language scaffolds"]
end
FCL --> SAL
FCL --> ICL
FCL --> CDL
FCL --> ISL
CL -->|constrains| SAL
SAL --> ICL
ICL --> CDL
CDL --> ISL
classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px;
classDef alert fill:#ffebee,stroke:#c62828,color:#b71c1c,stroke-width:1.5px;
class FCL,SAL,ICL,CDL,ISL normative;
class CL alert;
```

**Interpretation** `CL` does not replace functional intent; it injects explicit technology, hardware, and infrastructure bounds into `SAL` , and that constrained architecture then propagates into contracts, design, and scaffolding.

**Authority basis** `tier_definitions` , `constraint_precedence` , and the tier-local parent/child relationship tables in Section 4 .

**4.1 XPD - Existential Purpose Document**

| PROPERTY                                  | VALUE                                                                                                                                                         |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Representative node                       | XPD-0.1                                                                                                                                                       |
| Label                                     | Existential Purpose Document                                                                                                                                  |
| Core question                             | What human or societal need does this system exist to address, and what ethical boundaries govern all downstream decisions?                                   |
| Optional                                  | Yes                                                                                                                                                           |
| Root / activation rule                    | Always root when active. Required when ethical impact is not none or societal scale exceeds personal; skippable for internal tooling with no external effect. |
| Parent relationships                      | NONE via derives under condition none_root                                                                                                                    |
| Child relationships                       | SIL via derives under condition always                                                                                                                        |
| Tier-specific schema or verification note | None beyond universal node contract                                                                                                                           |

| INCLUSION RULE | VERIFICATION_MODE | STATEMENT                                                                          |
|----------------|-------------------|------------------------------------------------------------------------------------|
| XPD-R1         | structural        | Must articulate a fundamental human or societal need being addressed.              |
| XPD-R2         | structural        | Must be immutable across the project lifecycle; changes require a new XPD version. |
| XPD-R3         | semantic          | Must be comprehensible to non-technical stakeholders without a                     |

|        |            |                                                                               |
|--------|------------|-------------------------------------------------------------------------------|
|        |            | glossary.                                                                     |
| XPD-R4 | structural | Must establish ethical boundary conditions all subsequent tiers must satisfy. |
| XPD-R5 | structural | Must define success criteria independent of implementation metrics.           |
| XPD-R6 | structural | Must identify populations who could be harmed and the safeguards required.    |

| EXCLUSION RULE | STATEMENT                                                                          |
|----------------|------------------------------------------------------------------------------------|
| XPD-E1         | Must not contain solution concepts, technology references, or architectural ideas. |
| XPD-E2         | Must not contain quantitative performance targets; those belong in GPCL.           |
| XPD-E3         | Must not contain regulatory or legal constraints; those belong in GPCL.            |

## 4.2 **SIL** - Strategic Intent Layer

| PROPERTY                                  | VALUE                                                                                        |
|-------------------------------------------|----------------------------------------------------------------------------------------------|
| Representative node                       | SIL-1.1                                                                                      |
| Label                                     | Strategic Intent Layer                                                                       |
| Core question                             | Why does this system exist, and what business outcomes must it achieve?                      |
| Optional                                  | No                                                                                           |
| Root / activation rule                    | Root when XPD is inactive                                                                    |
| Parent relationships                      | XPD <b>via</b> derives if XPD <b>active</b> ; NONE <b>via</b> derives if XPD <b>inactive</b> |
| Child relationships                       | GPCL <b>via</b> derives under condition <b>always</b>                                        |
| Tier-specific schema or verification note | None beyond universal node contract                                                          |

| INCLUSION<br>RULE | VERIFICATION_MODE | STATEMENT                                                                            |
|-------------------|-------------------|--------------------------------------------------------------------------------------|
| SIL-R1            | structural        | Must define the core business problem or opportunity being addressed.                |
| SIL-R2            | structural        | Must specify strategic objectives with measurable outcomes.                          |
| SIL-R3            | structural        | Must identify all stakeholder categories and their value propositions.               |
| SIL-R4            | structural        | Must establish explicit scope boundaries, including in-scope and out-of-scope areas. |
| SIL-R5            | structural        | Must define organizational success metrics.                                          |
| SIL-R6            | structural        | Must remain stable under technology changes.                                         |

| EXCLUSION<br>RULE | STATEMENT                                                                              |
|-------------------|----------------------------------------------------------------------------------------|
| SIL-E1            | Must not reference hardware, technology stacks, frameworks, or languages.              |
| SIL-E2            | Must not contain regulatory mandates or compliance requirements; those belong in GPCL. |
| SIL-E3            | Must not prescribe architectural patterns or implementation strategies.                |
| SIL-E4            | Must not contain quantitative performance metrics; those belong in GPCL.               |

#### 4.3 GPCL - Governance, Policy & Quality Layer

| PROPERTY            | VALUE    |
|---------------------|----------|
| Representative node | GPCL-2.1 |



|                                           |                                                                                                                                          |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Label                                     | Governance, Policy & Quality Layer                                                                                                       |
| Core question                             | What non-negotiable external mandates, regulatory obligations, policy constraints, and measurable quality thresholds govern this system? |
| Optional                                  | No                                                                                                                                       |
| Root / activation rule                    | Never root; derives from <code>SIL</code>                                                                                                |
| Parent relationships                      | <code>SIL</code> <b>via</b> <code>derives</code> <b>under condition</b> <code>always</code>                                              |
| Child relationships                       | <code>FCL</code> <b>via</b> <code>derives</code> <b>under condition</b> <code>always</code>                                              |
| Tier-specific schema or verification note | No extra schema fields beyond the universal node contract                                                                                |

| INCLUSION RULE | VERIFICATION_MODE | STATEMENT                                                                                                             |
|----------------|-------------------|-----------------------------------------------------------------------------------------------------------------------|
| GPCL-R1        | structural        | Must enumerate all applicable regulatory frameworks with jurisdiction and scope.                                      |
| GPCL-R2        | semantic          | Must specify enforceable, testable constraints rather than aspirational targets.                                      |
| GPCL-R3        | structural        | Must identify contractual obligations imposed by third-party relationships.                                           |
| GPCL-R4        | structural        | Must define data sovereignty and residency requirements.                                                              |
| GPCL-R5        | structural        | Must specify audit and record-retention mandates.                                                                     |
| GPCL-R6        | structural        | Must specify quantifiable performance targets: latency, throughput, and concurrency ceilings.                         |
| GPCL-FCL-BR1   | semantic          | Every <code>GPCL-R6</code> target needs a corresponding <code>FCL</code> node that provides behavioral context rather |

|          |            |                                                                                                                                                                    |
|----------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|          |            | than merely repeating the number. If no user-facing behavioral dimension exists, the author must log <code>MISSING_MEDIATOR</code> in the reconciliation manifest. |
| GPCL-R7  | structural | Must specify reliability and availability targets such as SLAs, RTO, and RPO.                                                                                      |
| GPCL-R8  | structural | Must specify security requirements in technology-neutral language.                                                                                                 |
| GPCL-R9  | structural | Must specify scalability and accessibility requirements.                                                                                                           |
| GPCL-R10 | structural | Must cite parent <code>SIL</code> IDs for each constraint.                                                                                                         |

| EXCLUSION RULE | STATEMENT                                                                                   |
|----------------|---------------------------------------------------------------------------------------------|
| GPCL-E1        | Must not specify technology frameworks, library choices, or hardware specifications.        |
| GPCL-E2        | Must not describe functional system behaviors; those belong in <code>FCL</code> .           |
| GPCL-E3        | Must not contain business objectives or success metrics; those belong in <code>SIL</code> . |

#### 4.4 `FCL` - Functional Capability Layer

| PROPERTY               | VALUE                                                                                      |
|------------------------|--------------------------------------------------------------------------------------------|
| Representative node    | <code>FCL-3.1</code>                                                                       |
| Label                  | Functional Capability Layer                                                                |
| Core question          | What externally observable behaviors and user-facing capabilities must the system provide? |
| Optional               | No                                                                                         |
| Root / activation rule | Never root; derives from <code>GPCL</code>                                                 |
| Parent relationships   | <code>GPCL</code> via <code>derives</code> under condition <code>always</code>             |

|                                           |                                                                                       |
|-------------------------------------------|---------------------------------------------------------------------------------------|
| Child relationships                       | SAL via derives always; CL via derives if CL is active                                |
| Tier-specific schema or verification note | FCL-R7 adds mandatory logical data-entity enumeration for data-modifying capabilities |

| INCLUSION<br>RULE | VERIFICATION_MODE | STATEMENT                                                                                                                                                                                                                           |
|-------------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FCL-R1            | semantic          | Must describe capabilities from the perspective of a user or external system.                                                                                                                                                       |
| FCL-R2            | semantic          | Must specify user workflows end-to-end without naming components, classes, or modules.                                                                                                                                              |
| FCL-R3            | structural        | Must define event-driven behaviors and conditional business-logic rules.                                                                                                                                                            |
| FCL-R4            | structural        | Must specify user-observable state transitions and error conditions.                                                                                                                                                                |
| FCL-R5            | structural        | Must be decomposable into sub-capabilities for complex features.                                                                                                                                                                    |
| FCL-R6            | structural        | Must cite parent GPCL IDs for capabilities that satisfy governance or quality requirements.                                                                                                                                         |
| FCL-R7            | semantic          | For any capability that creates, reads, updates, or deletes persistent data, must enumerate all logical data entities involved and their CRUD relationship, without attribute typing, storage structures, keys, or integrity rules. |

| EXCLUSION<br>RULE | STATEMENT                                                                   |
|-------------------|-----------------------------------------------------------------------------|
| FCL-E1            | Must not name specific classes, modules, APIs, or algorithms.               |
| FCL-E2            | Must not specify network protocols, serialization formats, or data schemas. |
| FCL-E3            | Must not specify hardware requirements or infrastructure                    |

topology.

#### 4.5 CL - Constraint Layer

| PROPERTY                                  | VALUE                                                                                                                                |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Representative node                       | CL-4.1                                                                                                                               |
| Label                                     | Constraint Layer                                                                                                                     |
| Core question                             | What are the declared technology selections, hardware envelopes, and infrastructure ceilings that bound the system's implementation? |
| Optional                                  | Yes                                                                                                                                  |
| Root / activation rule                    | Active when technology, hardware, or infrastructure constraints are non-negotiable                                                   |
| Parent relationships                      | FCL via derives if CL is active                                                                                                      |
| Child relationships                       | SAL via constrains if CL is active                                                                                                   |
| Tier-specific schema or verification note | constraint_origin is required and branches verification between CL-R9 and CL-R9-imposed                                              |

| INCLUSION RULE | VERIFICATION_MODE | STATEMENT                                                                                |
|----------------|-------------------|------------------------------------------------------------------------------------------|
| CL-R1          | structural        | Must declare approved programming languages with version constraints.                    |
| CL-R2          | structural        | Must declare mandatory frameworks and core libraries with minimum version bounds.        |
| CL-R3          | structural        | Must declare required external service contracts without internal implementation detail. |
| CL-R4          | structural        | Must declare runtime environment constraints such as OS, container                       |

|               |            |                                                                                                                             |
|---------------|------------|-----------------------------------------------------------------------------------------------------------------------------|
|               |            | runtime, or execution environment.                                                                                          |
| CL-R5         | structural | Must explicitly declare prohibited technologies with rationale.                                                             |
| CL-R6         | structural | Must declare hardware envelopes when applicable, including CPU class, RAM floor, storage, and GPU.                          |
| CL-R7         | structural | Must declare infrastructure ceilings when applicable, including compute budget, storage cap, and bandwidth cap.             |
| CL-R8         | structural | Must specify deployment topology declarations such as on-premise, cloud-agnostic, hybrid, or edge.                          |
| CL-R9         | structural | Must cite FCL IDs for each derived constraint.                                                                              |
| CL-R9-imposed | structural | Must cite the external authority source for each imposed constraint. FCL citation becomes optional contextual traceability. |
| CL-R10        | structural | Must explicitly document internal reconciliations of conflicting hardware and technology constraints.                       |

| EXCLUSION RULE | STATEMENT                                                                                  |
|----------------|--------------------------------------------------------------------------------------------|
| CL-E1          | Must not auto-derive, infer, or recommend configurations; inference belongs to Extensions. |
| CL-E2          | Must not contain functional system behaviors; those belong in FCL.                         |
| CL-E3          | Must not contain cost models or TCO calculations; those belong in Extensions.              |

#### 4.6 SAL - System Architecture Layer

| PROPERTY | VALUE |
|----------|-------|
|          |       |

|                                           |                                                                                            |
|-------------------------------------------|--------------------------------------------------------------------------------------------|
| Representative node                       | SAL-5.1                                                                                    |
| Label                                     | System Architecture Layer                                                                  |
| Core question                             | How is the system structurally decomposed, and what patterns govern component interaction? |
| Optional                                  | No                                                                                         |
| Root / activation rule                    | Never root; merge node between functional derivation and optional constraint input         |
| Parent relationships                      | FCL via derives always; CL via constrains if CL is active                                  |
| Child relationships                       | ICL via derives under condition always                                                     |
| Tier-specific schema or verification note | SAL is the only valid merge node in the Core topology                                      |

| INCLUSION<br>RULE | VERIFICATION_MODE | STATEMENT                                                                                        |
|-------------------|-------------------|--------------------------------------------------------------------------------------------------|
| SAL-R1            | semantic          | Must define the overarching architectural pattern or patterns with rationale.                    |
| SAL-R2            | structural        | Must specify system decomposition into major subsystems with ownership boundaries.               |
| SAL-R3            | structural        | Must specify inter-subsystem communication patterns.                                             |
| SAL-R4            | structural        | Must specify concurrency model and data ownership rules.                                         |
| SAL-R5            | structural        | Must specify failure isolation and resilience boundaries.                                        |
| SAL-R6            | structural        | Must cite all active parent IDs for each major architectural decision: FCL, plus CL when active. |

| EXCLUSION<br>RULE | STATEMENT                                                                        |
|-------------------|----------------------------------------------------------------------------------|
| SAL-E1            | Must not contain exact data schemas or payload definitions; those belong in ICL. |

|        |                                                                                                               |
|--------|---------------------------------------------------------------------------------------------------------------|
| SAL-E2 | Must not contain class-level component blueprints; those belong in CDL.                                       |
| SAL-E3 | Must not contain executable code, algorithm implementations, or procedural logic; those belong in CDL or ISL. |

#### 4.7 ICL - Interface & Contracts Layer

| PROPERTY                                  | VALUE                                                                                                |
|-------------------------------------------|------------------------------------------------------------------------------------------------------|
| Representative node                       | ICL-6.1                                                                                              |
| Label                                     | Interface & Contracts Layer                                                                          |
| Core question                             | What are the formal, machine-verifiable contracts governing data exchange between system boundaries? |
| Optional                                  | No                                                                                                   |
| Root / activation rule                    | Never root; derives from SAL                                                                         |
| Parent relationships                      | SAL via <code>derives</code> under condition <code>always</code>                                     |
| Child relationships                       | CDL via <code>implements</code> under condition <code>always</code>                                  |
| Tier-specific schema or verification note | ICL-R2 requires machine-parseable schemas                                                            |

| INCLUSION RULE | VERIFICATION_MODE | STATEMENT                                                                                          |
|----------------|-------------------|----------------------------------------------------------------------------------------------------|
| ICL-R1         | structural        | Must define all inter-component and external API contracts with complete input and output schemas. |
| ICL-R2         | structural        | All schemas must be machine-parseable, such as JSON Schema, Protobuf, OpenAPI, or equivalent.      |
| ICL-R3         | structural        | Must specify serialization formats, encoding standards, and wire protocols per contract.           |
| ICL-R4         | structural        | Must specify mandatory fields, optional fields, type constraints,                                  |

|        |            |                                                                                                      |
|--------|------------|------------------------------------------------------------------------------------------------------|
|        |            | and validation rules.                                                                                |
| ICL-R5 | structural | Must specify error response contracts, including error codes, payload structure, and retry behavior. |
| ICL-R6 | structural | Must specify versioning strategy per contract.                                                       |
| ICL-R7 | structural | Must cite <code>SAL</code> IDs for each contract.                                                    |

| EXCLUSION RULE | STATEMENT                                                                           |
|----------------|-------------------------------------------------------------------------------------|
| ICL-E1         | Must not contain internal component state management or business logic.             |
| ICL-E2         | Must not specify architectural routing patterns; those belong in <code>SAL</code> . |
| ICL-E3         | Must not contain class or module blueprints; those belong in <code>CDL</code> .     |

#### 4.8 `CDL` - Component Design Layer

| PROPERTY                                  | VALUE                                                                                                                           |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Representative node                       | <code>CDL-7.1</code>                                                                                                            |
| Label                                     | Component Design Layer                                                                                                          |
| Core question                             | What are the structural blueprints of individual components, including public interfaces, internal state, and responsibilities? |
| Optional                                  | No                                                                                                                              |
| Root / activation rule                    | Never root; implements <code>ICL</code>                                                                                         |
| Parent relationships                      | <code>ICL</code> via <code>implements</code> under condition <code>always</code>                                                |
| Child relationships                       | <code>ISL</code> via <code>implements</code> under condition <code>always</code>                                                |
| Tier-specific schema or verification note | <code>CDL-R7</code> requires language-specific blueprints when <code>CL</code> declares multiple targets                        |



| INCLUSION<br>RULE | VERIFICATION_MODE | STATEMENT                                                                                                              |
|-------------------|-------------------|------------------------------------------------------------------------------------------------------------------------|
| CDL-R1            | structural        | Must define component names, logical responsibilities, and ownership boundaries.                                       |
| CDL-R2            | structural        | Must specify all public method or function signatures, including names, parameter types, return types, and exceptions. |
| CDL-R3            | structural        | Must specify internal state structures as a logical model rather than executable implementation.                       |
| CDL-R4            | structural        | Must specify component dependencies, including consumed components and <code>ICL</code> contracts.                     |
| CDL-R5            | structural        | Must map each component to the <code>ICL</code> contracts it implements.                                               |
| CDL-R6            | structural        | Must specify initialization, lifecycle, and teardown contracts for stateful components.                                |
| CDL-R7            | structural        | When <code>CL</code> declares multiple target languages, must produce language-specific blueprints for each target.    |

| EXCLUSION<br>RULE | STATEMENT                                                                               |
|-------------------|-----------------------------------------------------------------------------------------|
| CDL-E1            | Must not contain executable code bodies or algorithm implementations.                   |
| CDL-E2            | Must not contain system-wide architectural patterns; those belong in <code>SAL</code> . |
| CDL-E3            | Must not contain data serialization schemas; those belong in <code>ICL</code> .         |

## 4.9 ISL - Implementation Scaffold Layer

|  |  |
|--|--|
|  |  |
|--|--|

| PROPERTY                                  | VALUE                                                                                               |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Representative node                       | ISL-8.1                                                                                             |
| Label                                     | Implementation Scaffold Layer                                                                       |
| Core question                             | What is the minimal, structurally valid, traceable scaffolding required to initiate implementation? |
| Optional                                  | No                                                                                                  |
| Root / activation rule                    | Terminal leaf tier                                                                                  |
| Parent relationships                      | CDL <code>via</code> implements under condition <code>always</code>                                 |
| Child relationships                       | none                                                                                                |
| Tier-specific schema or verification note | ISL is the only valid leaf tier in a CLEAN Core DAG                                                 |

| INCLUSION<br>RULE | VERIFICATION_MODE | STATEMENT                                                                                                     |
|-------------------|-------------------|---------------------------------------------------------------------------------------------------------------|
| ISL-R1            | structural        | Must produce syntactically valid structural scaffolding in the target language.                               |
| ISL-R2            | structural        | Must embed docstrings or code comments with explicit parent DDR node IDs.                                     |
| ISL-R3            | structural        | Must include implementation hints as structured comments.                                                     |
| ISL-R4            | structural        | Must define all function or method bodies exclusively as stubs.                                               |
| ISL-R5            | structural        | Must be language-specific, with one ISL node per target language or runtime when multiple are declared in CL. |
| ISL-R6            | structural        | Must cite CDL parent IDs for every stub.                                                                      |

| EXCLUSION<br>RULE | STATEMENT |
|-------------------|-----------|
|                   |           |

|        |                                                                            |
|--------|----------------------------------------------------------------------------|
| ISL-E1 | Must not contain business logic or complete algorithmic logic.             |
| ISL-E2 | Must not contain infrastructure configuration; that belongs in Extensions. |

See also: Section 3, Section 5, Section 7.

## 5. Lifecycle and Operations

This section covers the operational state machine of DDR v6.3, including statuses, valid transitions, guards, canonical operations, DIRTY behavior, and conflict-resolution flow. Use Section 6 for Express Mode execution rules, Section 7 for reconciliation and CLEAN-state implications, and Section 9 for the schema-side lifecycle contract.

### 5.1 Status model and lifecycle authority

`lifecycle.status_transitions` is the sole normative authority for valid node status transitions.

| STATUS            | OPERATIONAL MEANING                                                    |
|-------------------|------------------------------------------------------------------------|
| DRAFT             | Authored but not yet activated through the validation path             |
| ACTIVE            | Structurally valid, review-complete, and current                       |
| DIRTY             | Requires re-validation because of direct change or upstream change     |
| DEPRECATED        | Still present but marked for retirement or replacement                 |
| SUPERSEDED        | Replaced but retained for audit lineage                                |
| SUPERSEDE_PENDING | Transient state during an in-flight <code>SUPERSEDE</code> transaction |

Figure 5.1. Lifecycle state machine and rollback authority

%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% stateDiagram  
accTitle: Lifecycle state machine and rollback authority accDescr: Shows the closed DDR node status machine, including SUPERSEDE\_PENDING as a transactional state that can commit or roll back to the recorded prior\_status. direction LR [\*] --> DRAFT DRAFT --> ACTIVE: VALIDATE / gc-001, gc-005 ACTIVE --> DIRTY: MODIFY ACTIVE --> DEPRECATED: MODIFY / gc-002 ACTIVE --> SUPERSEDE\_PENDING: SUPERSEDE / gc-007 DIRTY --> ACTIVE: VALIDATE after VERIFY / gc-001, gc-005, gc-006 DIRTY --> DEPRECATED: MODIFY / gc-002 DIRTY --> SUPERSEDE\_PENDING: SUPERSEDE / gc-007 DEPRECATED --> ACTIVE: MODIFY / gc-002, gc-003, gc-004 DEPRECATED --> SUPERSEDE\_PENDING: SUPERSEDE / gc-007 SUPERSEDE\_PENDING --> SUPERSEDED: SUPERSEDE commit / gc-008 SUPERSEDE\_PENDING --> ROLLBACK: SUPERSEDE rollback / gc-009 state ROLLBACK <> ROLLBACK --> ACTIVE: prior\_status = ACTIVE ROLLBACK --> DIRTY: prior\_status = DIRTY ROLLBACK --> DEPRECATED: prior\_status = DEPRECATED note right of SUPERSEDE\_PENDING prior\_status must be recorded before commit or rollback end note classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px; classDef transient fill:#fff3e0,stroke:#ef6c00,color:#bf360c,stroke-width:1.5px; class DRAFT,ACTIVE,DIRTY,DEPRECATED,SUPERSEDED normative; class SUPERSEDE\_PENDING transient;

**Interpretation** DDR v6.3 treats lifecycle changes as a closed machine in which `SUPERSEDE` is transactional and rollback is resolved only through the recorded `prior_status` branch.

**Authority basis** `lifecycle.status_transitions`, `lifecycle.guard_definitions`, and Sections 5.1-5.2.

**Schema note** The rollback row is modeled in the schema as `to_node_field: prior_status`, not as three separate transition rows. The diagram expands the restored target states for readability.

5.2 Status transitions

| FROM   | TO     | OPERATION | PHASE / SIDE-EFFECT | PREREQUI |
|--------|--------|-----------|---------------------|----------|
| DRAFT  | ACTIVE | VALIDATE  | none                | none     |
| ACTIVE | DIRTY  | MODIFY    | none                | none     |
| ACTIVE | DIRTY  | MODIFY    | side_effect:        | none     |

|                   |                   |           |                    |        |
|-------------------|-------------------|-----------|--------------------|--------|
|                   |                   |           | propagation        |        |
| ACTIVE            | DEPRECATED        | MODIFY    | none               | none   |
| ACTIVE            | SUPERSEDE_PENDING | SUPERSEDE | none               | none   |
| DIRTY             | ACTIVE            | VALIDATE  | none               | VERIFY |
| DIRTY             | DEPRECATED        | MODIFY    | none               | none   |
| DIRTY             | SUPERSEDE_PENDING | SUPERSEDE | none               | none   |
| DEPRECATED        | SUPERSEDE_PENDING | SUPERSEDE | none               | none   |
| SUPERSEDE_PENDING | SUPERSEDED        | SUPERSEDE | phase:<br>commit   | none   |
| SUPERSEDE_PENDING | prior_status      | SUPERSEDE | phase:<br>rollback | none   |
| DEPRECATED        | ACTIVE            | MODIFY    | none               | none   |

### 5.3 Guard definitions

| GUARD  | VERIFICATION_MODE | DESCRIPTION                                        |
|--------|-------------------|----------------------------------------------------|
| gc-001 | <b>STRUCTURAL</b> | All structural rules for the node pass validation. |

|        |            |                                                                                                                                                                                                                                  |
|--------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| gc-002 | MANUAL     | Deprecation rationale is explicitly documented.                                                                                                                                                                                  |
| gc-003 | MANUAL     | Any previously set deprecation sunset date is cleared.                                                                                                                                                                           |
| gc-004 | MANUAL     | Status reversal is logged in the reconciliation manifest.                                                                                                                                                                        |
| gc-005 | STRUCTURAL | All review items are resolved.                                                                                                                                                                                                   |
| gc-006 | STRUCTURAL | Per-node validation scope is explicitly confirmed.                                                                                                                                                                               |
| gc-007 | STRUCTURAL | Before entering <code>SUPERSEDE_PENDING</code> , the node's current status must be recorded in <code>prior_status</code> , and that value must be one of <code>ACTIVE</code> , <code>DEPRECATED</code> , or <code>DIRTY</code> . |
| gc-008 | STRUCTURAL | Replacement node is successfully inserted and validated, children are rewired to the replacement ID, affected children are set <code>DIRTY</code> , and <code>prior_status</code> is cleared.                                    |
| gc-009 | STRUCTURAL | Replacement insert failed or child rewiring failed; source node reverts to <code>prior_status</code> , replacement is removed if necessary, and <code>SUPERSEDE_FAILED</code> is logged.                                         |

Guards are declared only as **STRUCTURAL** or **MANUAL** checks. Separate node-level `VALIDATE` logic may also emit **SEMANTIC** `REVIEW_REQUIRED` items for tier-local rule review before activation.

Figure 5.2. Verification-mode handling across guards and rule review

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: Verification-mode handling across guards and rule review accDescr: Separates lifecycle guard verification modes from the independent semantic REVIEW_REQUIRED path emitted by VALIDATE or VERIFY.
subgraph GUARDS["Lifecycle guard_definitions"]
CHECK["GuardDefinition"] --> KIND["verification_mode"]
KIND -->|structural| AUTO["Mechanical pass/fail gate"]
KIND -->|manual| HUMAN["Recorded human action or rationale"]
end
subgraph REVIEW["Separate semantic review path"]
VALID["VALIDATE or VERIFY finding"] --> SEM["Semantic tier rule or cross-node review"]
SEM --> RR["Emit REVIEW_REQUIRED item"]
RR --> DISP["Human disposition recorded before activation or CLEAN"]
end
AUTO --> OUT["Transition or activation decision"]
HUMAN --> OUT
DISP -. resolution required .-> OUT
classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px;
classDef schema fill:rgba(59,130,246,0.1),stroke:rgba(59,130,246,0.5),color:#60a5fa,stroke-width:2px;
classDef caution fill:rgba(245,158,11,0.1),stroke:rgba(245,158,11,0.5),color:#fbbf24,stroke-width:2px;
class CHECK,KIND,AUTO,VALID,SEM,RR schema;
class HUMAN,DISP caution;
class OUT normative;
```

**Interpretation** DDR separates lifecycle guard verification modes from semantic review obligations so that `guard_definitions` remain closed to `structural` and `manual` while semantic review still stays explicit.

**Authority basis** `lifecycle.guard_definitions`, the `VALIDATE` operation contract, and the `verification_mode` glossary definition.

5.4 Canonical operations

| OPERATION | MUTATES DAG | SOURCE-DEFINED BEHAVIOR                                                                                                                 | HIGH-SIGNAL VALIDATION AND OUTPUT DETAILS                                                                 |
|-----------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| INSERT    | Yes         | Create a node with auto-assigned ID, <code>parent_ids</code> , and tier-compliant content. Supports both forward and reverse direction. | Full atomic ruleset validation, parent existence, and DAG cycle detection.                                |
| DELETE    | Yes         | Remove a node and cascade orphan detection to children.                                                                                 | Former children become <code>DIRTY</code> ; zero-parent cases must resolve through re-attachment, cascade |



|                  |     |                                                                                                                                                                          |                                                                                                                                                                                                                                                        |
|------------------|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  |     |                                                                                                                                                                          | delete, or replacement; manifest is updated.                                                                                                                                                                                                           |
| MODIFY           | Yes | Update node content and increment version.                                                                                                                               | Re-validates the ruleset, re-checks citations, and propagates <code>DIRTY</code> to all descendants.                                                                                                                                                   |
| SUPERSEDE        | Yes | Replace a node transactionally while preserving the old ID for audit lineage.                                                                                            | Enters <code>SUPERSEDE_PENDING</code> , attempts replacement <code>INSERT</code> , re-wires child <code>parent_ids</code> on commit, sets immediate children <code>DIRTY</code> , or rolls back atomically with <code>SUPERSEDE_FAILED</code> logging. |
| VERIFY           | No  | Traverse the DAG downward and validate citation chains, edge types, ID references, orphan conditions, contamination, and optional cross-node semantic consistency rules. | Returns <code>CLEAN</code> or <code>DIRTY</code> with itemized structural findings; may also emit non-blocking <code>REVIEW_REQUIRED</code> items for semantic consistency review.                                                                     |
| VALIDATE         | No  | Check one node against its tier's full atomic ruleset.                                                                                                                   | Structural rules return pass/fail with violated rule IDs. Semantic rules emit <code>REVIEW_REQUIRED</code> items plus a <code>validation_scope</code> declaration that records what was evaluated.                                                     |
| UNBUNDLE_SCAN    | No  | Read-only pre-flight scan of an Express Mode group.                                                                                                                      | Produces one diagnostic object per fragment: <code>fragment_id</code> , <code>content_preview</code> , <code>detected_annotation</code> , <code>confidence</code> , and <code>ambiguity_reason</code> when confidence is not <code>high</code> .       |
| UNBUNDLE_EXECUTE | Yes | Atomic commit-phase expansion of an Express Mode group                                                                                                                   | Succeeds only when every fragment is confidently assignable                                                                                                                                                                                            |

|  |  |                                   |                                                                                              |
|--|--|-----------------------------------|----------------------------------------------------------------------------------------------|
|  |  | into constituent full-mode tiers. | or explicitly deferred. Rejection payload is the complete <code>UNBUNDLE_SCAN</code> result. |
|--|--|-----------------------------------|----------------------------------------------------------------------------------------------|

**Figure 5.3. Canonical operation families**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: Canonical operation families accDescr: Groups the closed DDR operation namespace into mutation, verification, and Express-mode families. OPS["Canonical operations"] subgraph
MUT["Graph mutation"] MUTH["Mutation family"] INSERT["INSERT"] DELETE["DELETE"]
MODIFY["MODIFY"] SUPERSEDE["SUPERSEDE"] MUTH --> INSERT MUTH --> DELETE MUTH -->
MODIFY MUTH --> SUPERSEDE end subgraph CHECKH["Validation and verification"]
CHECKH["Verification family"] VERIFY["VERIFY"] VALIDATE["VALIDATE"] CHECKH --> VERIFY CHECKH -->
VALIDATE end subgraph EXPH["Express expansion"] EXPH["Express family"]
SCAN["UNBUNDLE_SCAN"] EXEC["UNBUNDLE_EXECUTE"] EXPH --> SCAN EXPH --> EXEC end OPS -->
MUTH OPS --> CHECKH OPS --> EXPH classDef normative
fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px; classDef schema
fill:rgba(59,130,246,0.1),stroke:rgba(59,130,246,0.5),color:#60a5fa,stroke-width:2px; classDef
caution fill:rgba(245,158,11,0.1),stroke:rgba(245,158,11,0.5),color:#fbbf24,stroke-width:2px; class
OPS,MUTH,CHECKH,EXPH normative; class INSERT,DELETE,MODIFY,SUPERSEDE caution; class
VERIFY,VALIDATE,SCAN,EXEC schema;
```

**Interpretation** The operation namespace is intentionally closed: mutation, validation, and Express expansion each have named entry points rather than informal aliases or mixed phase tokens.

**Authority basis** `operations.core_operations` and the normalization changes summarized in Section 2.3.

Operational nuances that are easy to miss if you only read the names:

- `INSERT` can run as validated insertion that yields an `ACTIVE` node synchronously, or as draft insertion via `validate=false` override that yields a `DRAFT` node.
- `VALIDATE` is node-local and tier-local; `VERIFY` is graph-traversal-oriented and may consider optional cross-node semantic consistency rules.
- `UNBUNDLE_SCAN` is independently invocable and is not just an internal pre-step of `UNBUNDLE_EXECUTE`.
- `UNBUNDLE_EXECUTE` does not invent placement for unclear fragments. It either gets deterministic allocation or rejects atomically unless the fragment is explicitly deferred.

**Example** `UNBUNDLE_SCAN` diagnostic payload:

```
fragment_id: G2-F03
content_preview: "[CL] Runtime must be Python 3.10+ and run in Linux containers."
detected_annotation: CL
confidence: high
ambiguity_reason: null
```

5.5 DIRTY triggers and classification

| TRIGGER                                                                                   | NODES AFFECTED                                             |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------|
| Node modified                                                                             | Modified node plus all descendants                         |
| Node deleted                                                                              | All former children of the deleted node                    |
| Parent becomes <code>SUPERSEDED</code> and child <code>parent_ids</code> are auto-updated | Immediate children only; grandchildren do not auto-cascade |
| <code>CL</code> constraint added or modified                                              | <code>SAL</code> plus all <code>SAL</code> descendants     |
| <code>XPD</code> ethical boundary modified                                                | All tiers                                                  |

| DIRTY CLASSIFICATION | MEANING                                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>STRUCTURAL</b>    | Structural change such as parent rewiring without immediate proof of semantic invalidation                                   |
| <b>SEMANTIC</b>      | Probable semantic invalidation requiring downstream review or content change before <code>CLEAN</code> can be re-established |

The specification also states the following `SUPERSEDE` `DIRTY` behavior:

- Child nodes affected by parent rewiring enter `DIRTY` with classification `structural`.
- Structural `DIRTY` does not automatically propagate to descendants.
- If later validation or modification reveals content drift, the affected node's `DIRTY` condition is reclassified as `semantic`, and normal downstream propagation resumes.

**Figure 5.4. DIRTY propagation and reclassification workflow**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: DIRTY propagation and reclassification workflow accDescr: Distinguishes structural DIRTY from semantic DIRTY and shows the scoped supersede exception before revalidation. TRIG["Trigger event"] --> MARK["Mark impacted nodes DIRTY"] MARK --> CLASS["DIRTY classification"] subgraph STRUCT["Structural DIRTY path"] LOCAL["Structural DIRTY: rewiring or topology impact only"] SCOPED["Scoped supersede rule: immediate children only"] RECHECK["VERIFY or VALIDATE"] LOCAL --> SCOPED --> RECHECK end subgraph SEMANTIC["Semantic DIRTY path"] CASCADE["Semantic DIRTY: probable content drift"] PROP["Propagate review downstream"] REPAIR["VERIFY, VALIDATE, or MODIFY"] CASCADE --> PROP --> REPAIR end CLASS --> |structural| LOCAL CLASS --> |semantic| CASCADE RECHECK --> RESOLVE["Return to ACTIVE or remain DIRTY"] REPAIR --> RESOLVE classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px; classDef schema fill:rgba(59,130,246,0.1),stroke:rgba(59,130,246,0.5),color:#60a5fa,stroke-width:2px; classDef caution fill:rgba(245,158,11,0.1),stroke:rgba(245,158,11,0.5),color:#fbbf24,stroke-width:2px; class TRIG,MARK,CLASS schema; class LOCAL,SCOPED,RECHECK,RESOLVE normative; class CASCADE,PROP,REPAIR caution;
```

**Interpretation** DDR distinguishes structural rewiring fallout from semantic drift so that not every topology change triggers blind full-depth propagation.

**Authority basis** `operations.dirty_flag_triggers`, `dirty_classification`, `supersede_dirty_behavior`, and Section 5.5.

Authoritative DIRTY notes that materially affect implementation behavior:

- A `DRAFT` node created through draft insertion is structurally present in the DAG but excluded from CLEAN compliance checks until successfully validated.
- While a node is in `SUPERSEDE_PENDING`, `VERIFY` treats that condition as a blocking manifest item rather than as ordinary `DIRTY` propagation.
- `SUPERSEDE` commit marks immediate children `DIRTY` with classification `structural`; grandchildren do not auto-cascade from the rewiring alone.
- `SUPERSEDE` rollback restores the source node to `prior_status` and produces no `DIRTY` side-effects.
- If a rewired child is later modified during re-validation, ordinary `MODIFY` cascade rules apply from that child downward.
- `DEPRECATED` nodes remain structurally traversable and are still part of `VERIFY` scope; they are not removed from the graph merely by deprecation.

## 5.6 Conflict resolution and resolution workflow

Conflict resolution protocol:

1. Identify conflicting nodes and violated rules.
2. Classify the conflict as logical, physical, or semantic.
3. Escalate to the designated authoring authority.
4. Record the resolution decision and rationale.

5. Apply `MODIFY`, `SUPERSEDE`, or `DELETE` as required.

Audit requirement:

- All conflict resolutions must be recorded in the reconciliation manifest with before-and-after state references and disposition authority.

Resolution workflow:

```
DETECT CHANGE -> SET DIRTY -> SCAN DOWNSTREAM -> GENERATE PENDING ITEMS  
-> EXECUTE OPERATION -> VERIFY -> SET CLEAN OR REPEAT
```

**Figure 5.5. `SUPERSEDE` transaction with commit and rollback**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor':  
'#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a',  
'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize':  
'14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% sequenceDiagram  
accTitle: SUPERSEDE transaction with commit and rollback accDescr: Shows the transactional  
SUPERSEDE sequence, including replacement insertion, child rewiring, DIRTY marking, rollback,  
and the rule that partial rewiring is forbidden. participant A as Authoring action participant S as  
Source node participant R as Replacement node participant C as Child nodes participant M as  
Manifest Note over S,R: No partial child rewiring is permitted. A->>S: SUPERSEDE request S->>S:  
Set SUPERSEDE_PENDING and record prior_status S->>R: INSERT replacement candidate alt  
Replacement validates and rewiring succeeds R->>C: Rewire parent_ids to replacement ID C->>C:  
Set DIRTY on affected children S->>S: Clear prior_status and set SUPERSEDED else INSERT fails or  
rewiring fails S->>R: Discard replacement if created S->>S: Revert to prior_status S->>M: Log  
SUPERSEDE_FAILED Note over S,C: Children remain unchanged on rollback. end
```

**Interpretation** `SUPERSEDE` is not a rename; it is a transactional replace-and-rewire operation with an explicit rollback path that forbids partial child rewiring.

**Authority basis** `operations.core_operations.SUPERSEDE`,  
`lifecycle.status_transitions`, `gc-007`, `gc-008`, and `gc-009`.

See also: Section 6, Section 7, Section 9.

# 6. Consumption Modes and Express Mode

This section covers the two declared consumption modes, the fixed four-group Express Mode structure, and the deterministic UNBUNDLE\_SCAN / UNBUNDLE\_EXECUTE contract. Use Section 5 for the underlying operation model and Section 9 for the express-profile schema rules.

## 6.1 Consumption modes

| MODE                                           | DESCRIPTION                                                                                                 | BEST FIT                                  |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <div>EXPRESS<br/>(4<br/>GROUPS)</div>          | Adjacent tiers are bundled into fixed groups and later expanded through UNBUNDLE_SCAN and UNBUNDLE_EXECUTE. | Small-to-medium projects                  |
| <div>FULL<br/>(ALL<br/>ACTIVE<br/>TIERS)</div> | Every active tier is specified independently.                                                               | Complex, regulated, or enterprise systems |

Figure 6.1. Full vs Express authority split

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: Full versus Express authority split
accDescr: Compares full-mode authoring with Express-mode grouping and the unbundle protocol that expands groups back into independent active tiers.
DDR["DDR consumption"]
subgraph FULL["Full mode"]
ALL["Independent active tiers"]
end
subgraph EXPRESS["Express mode"]
GROUPS["Fixed groups G1-G4"]
SCAN["UNBUNDLE_SCAN"]
EXEC["UNBUNDLE_EXECUTE"]
end
DDR --> ALL
DDR --> GROUPS
GROUPS --> SCAN
SCAN --> EXEC
SCAN --> ALL
classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px;
classDef caution fill:rgba(245,158,11,0.1),stroke:rgba(245,158,11,0.5),color:#fbbf24,stroke-width:2px;
class DDR,ALL normative;
class GROUPS,SCAN,EXEC caution;
```

**Interpretation** Express Mode is an alternate presentation and authoring path, not a reduced DDR model; successful unbundling lands back on the same full-tier surface.

**Authority basis** consumption\_modes , express\_mode.description , and Section 6.1 .

## 6.2 Express Mode groups

| GROUP | TIERS           | LABEL                          |
|-------|-----------------|--------------------------------|
| G1    | XPDL, SIL, GPCL | Purpose, Strategy & Governance |
| G2    | FCL, CL         | Capabilities & Constraints     |
| G3    | SAL, ICL        | Architecture & Contracts       |
| G4    | CDL, ISL        | Design & Scaffolding           |

**Figure 6.2. Fixed G1–G4 composition**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: Fixed G1 through G4 composition
accDescr: Shows the four immutable Express Mode groups and the exact tier members assigned to each group.
subgraph G1["G1 Purpose, Strategy, and Governance"]
G1H["G1"]
XPDL["XPDL"]
SIL["SIL"]
GPCL["GPCL"]
G1H --> XPDL
G1H --> SIL
G1H --> GPCL
end
subgraph G2["G2 Capabilities and Constraints"]
G2H["G2"]
FCL["FCL"]
CL["CL"]
G2H --> FCL
G2H --> CL
end
subgraph G3["G3 Architecture and Contracts"]
G3H["G3"]
SAL["SAL"]
ICL["ICL"]
G3H --> SAL
G3H --> ICL
end
subgraph G4["G4 Design and Scaffolding"]
G4H["G4"]
CDL["CDL"]
ISL["ISL"]
G4H --> CDL
G4H --> ISL
end
G1H --> G2H
G2H --> G3H
G3H --> G4H
classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px;
classDef caution fill:rgba(245,158,11,0.1),stroke:rgba(245,158,11,0.5),color:#fbbf24,stroke-width:2px;
class G1H,G2H,G3H,G4H caution;
class XPDL,SIL,GPCL,FCL,CL,SAL,ICL,CDL,ISL normative;
```

**Interpretation** Express grouping is fixed and closed; authors do not invent alternate bundles or reorder tiers inside the group system.

**Authority basis** `express_mode.groups` and Section 6.2 .

## 6.3 Express Mode contract

The authoritative `express_mode` block defines the following:

- Express Mode is not a reduced system; it is grouped presentation of the full model.
- `UNBUNDLE_SCAN` is a read-only pre-flight classifier.
- `UNBUNDLE_EXECUTE` is the commit-phase operation and the only canonical commit token in v6.3.
- On successful unbundling, `parent_ids` auto-wire to the immediately superior unbundled tier, satisfying `CIT-R2` without manual intervention.

**Figure 6.3.** UNBUNDLE\_SCAN / UNBUNDLE\_EXECUTE sequence

```
%%{init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% sequenceDiagram
accTitle: UNBUNDLE_SCAN and UNBUNDLE_EXECUTE sequence
accDescr: Shows the Express Mode scan and execute protocol, including deterministic allocation, deferred rationale recording, and rejection without mutation.
participant A as Author
participant G as Express group
node
  participant S as UNBUNDLE_SCAN
  participant M as Reconciliation manifest
  participant F as Full-mode tiers
  Note over S,F: Successful execute auto-wires parent_ids to the superior unbundled tier.
  A->>G: request scan
  G->>S: classify fragments alt every fragment is high or explicitly deferred
  S->>M: record deferred rationale if present
  A->>G: invoke UNBUNDLE_EXECUTE
  G->>F: create constituent tier nodes
  F->>F: auto-wire parent_ids else ambiguous or unassigned fragment remains undeferred
  S->>A: reject with full scan diagnostics
  G->>G: preserve source node without mutation
end
```

**Interpretation** Unbundling is deliberately split into a read-only diagnostic phase and an atomic commit phase so that ambiguous fragments cannot silently mutate the graph.

**Authority basis** `express_mode.description`, `unbundle_determinism_rule`, and the `UNBUNDLE_SCAN / UNBUNDLE_EXECUTE` operation definitions.

### 6.4 Deterministic unbundling and deferred fragments

The authoritative determinism rule requires:

- Explicit tier annotations inside grouped content when a group contains conditionally activatable tiers, specifically `G1` and `G2`
- Per-fragment classification into `high`, `ambiguous`, or `none`
- Rejection of `UNBUNDLE_EXECUTE` when any fragment remains ambiguous or unassigned without explicit deferment

Deferred-fragment handling requires:

- Explicit `[DEFER]` annotation
- Recorded human rationale in the reconciliation manifest
- Retention of deferred fragments in the source Express Mode group node
- Atomic rejection when ambiguous fragments are neither confidently classified nor explicitly deferred

Worked `G2` example:

| FRAGMENT                                                             | EXPECTED SCAN RESULT | REASONING                                              |
|----------------------------------------------------------------------|----------------------|--------------------------------------------------------|
| [FCL] User uploads an invoice PDF and sees validation errors inline. | high -> FCL          | User-observable behavior with explicit tier annotation |



|                                                                |                                       |                                                                                   |
|----------------------------------------------------------------|---------------------------------------|-----------------------------------------------------------------------------------|
| [CL] Runtime must be Python 3.10+ and run in Linux containers. | high -> CL                            | Declared technology and environment bounds with explicit tier annotation          |
| Uploads should be fast.                                        | none                                  | No tier annotation and insufficient deterministic allocation surface              |
| [DEFER] Retention wording pending legal review.                | deferred from current execute attempt | Explicit defer marker plus recorded rationale permits atomic progress on the rest |

Example rejection posture:

```
fragment_id: G2-F04
content_preview: "Uploads should be fast."
detected_annotation: null
confidence: none
ambiguity_reason: "No explicit [FCL] or [CL] marker was supplied."
```

If that fragment is not explicitly deferred, `UNBUNDLE_EXECUTE` must reject with the complete scan result and leave the source group node structurally unchanged.

**Figure 6.4. Deferred-fragment and atomic-rejection workflow**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: Deferred-fragment and atomic-rejection workflow accDescr: Shows how
UNBUNDLE_SCAN outcomes resolve into immediate execution, explicit deferral, or atomic
rejection. FRAG["Content fragment"] --> CLASS{"Classification"} subgraph ACCEPT["Accepted or
deferred outcomes"] READY["Eligible for UNBUNDLE_EXECUTE"] KEEP["Retain in source group
node"] MANIFEST["Record rationale in reconciliation manifest"] end subgraph REJECTPATH["Atomic
rejection path"] DECIDE["Explicit [DEFER]?"] REJECT["Reject UNBUNDLE_EXECUTE atomically"] end
CLASS -->|high| READY CLASS -->|ambiguous or none| DECIDE DECIDE -->|yes| KEEP --> MANIFEST
DECIDE -->|no| REJECT classDef normative
fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px; classDef caution
fill:rgba(245,158,11,0.1),stroke:rgba(245,158,11,0.5),color:#fbbf24,stroke-width:2px; classDef alert
fill:#ffebee,stroke:#c62828,color:#b71c1c,stroke-width:1.5px; class FRAG,CLASS,DECIDE caution;
class READY,KEEP,MANIFEST normative; class REJECT alert;
```

**Interpretation** Deferred fragments remain traceable and local to the original Express node, while undeferred ambiguity blocks the entire commit so that partial unbundling never becomes authoritative.

**Authority basis** `express_mode.unbundle_determinism_rule`, `express_mode.deferred_fragment_handling`, and Section 6.4.

See also: Section 5, Section 9.

# 7. Constraint Precedence, Reconciliation, and CLEAN State

This section covers how DDR v6.3 resolves conflicting constraints, tracks unresolved conditions, and determines whether a graph may be treated as CLEAN. Use Section 5 for the operation and DIRTY mechanics that feed reconciliation, Section 9 for the machine-validation surface, and Section 10 for glossary and historical appendix context.

## 7.1 Constraint precedence hierarchy

| PRIORITY | TIER | RATIONALE                                                                    |
|----------|------|------------------------------------------------------------------------------|
| 1        | XPD  | Ethical boundary conditions are inviolable.                                  |
| 2        | SIL  | Strategic intent defines the purpose of all design decisions.                |
| 3        | GPCL | External regulatory mandates and quality thresholds are non-negotiable.      |
| 4        | FCL  | Functional requirements operate within the constraint envelope.              |
| 5        | CL   | Technology, hardware, and infrastructure constraints are externally imposed. |
| 6        | SAL  | Architecture is bounded by all above.                                        |
| 7        | ICL  | Contracts derive from architecture.                                          |
| 8        | CDL  | Design derives from contracts.                                               |
| 9        | ISL  | Scaffolding derives from design.                                             |

**Figure 7.1. Constraint precedence ladder with physical-escalation branch**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart TD
accTitle: Constraint precedence ladder with physical-escalation branch
accDescr: Shows the ordered logical precedence ladder, the XPD veto branch, and the escalation path for imposed or physical CL conflicts.
subgraph LADDER["Logical precedence ladder"]
  XPD["1 XPD"] --> SIL["2 SIL"]
  GPCL["3 GPCL"] --> FCL["4 FCL"]
  CL["5 CL"] --> SAL["6 SAL"]
  ICL["7 ICL"] --> CDL["8 CDL"]
  ISL["9 ISL"]
end
ESC["Escalate to authoring authority"]
XPD --> SIL --> GPCL --> FCL --> CL --> SAL --> ICL --> CDL --> ISL
XPD -. ethical veto .-> ESC
CL -. imposed or physical conflict .-> ESC
classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px;
classDef alert fill:#ffebee,stroke:#c62828,color:#b71c1c,stroke-width:1.5px;
class XPD,SIL,GPCL,FCL,SAL,ICL,CDL,ISL normative;
class CL,ESC alert;
```

**Interpretation** The logical override ladder is linear, but it contains explicit non-override branches:

- `XPD` ethical boundaries and imposed or physically impossible
- `CL` constraints force escalation instead of silent precedence resolution.

**Authority basis** `constraint_precedence.tiers`, `override_principle`, `physical_constraint_rule`, and `physical_constraint_escalation`.

Override principle:

- Higher-priority tiers override lower-priority tiers.
- An `XPD` ethical boundary acts as an absolute veto right over downstream decisions.

**7.2 Constraint classes and escalation rules**

| CONSTRAINT CLASS | DESCRIPTION                                                                                                                                   |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| LOGICAL          | Governed by the formal tier precedence hierarchy                                                                                              |
| PHYSICAL         | Represents non-negotiable physical realities or externally imposed constraints that cannot be silently overridden by logical precedence alone |

Additional precedence rules:

- Intra-tier conflicts must be documented and resolved before conflicting nodes can become `ACTIVE`.
- Any `CL` node with `constraint_origin = imposed` is treated as a non-overridable physical-or-external constraint during precedence evaluation.
- Physical incompatibilities must be escalated to the authoring authority; precedence does not authorize silently overriding physical or externally imposed constraints.

7.3 Reconciliation manifest

Tracked values:

| TRACK                                   | MEANING                                   |
|-----------------------------------------|-------------------------------------------|
| Total node count by tier                | Current topology inventory                |
| ACTIVE, DIRTY, DRAFT, DEPRECATED counts | Status distribution                       |
| Pending items list                      | Unresolved review, gap, and failure items |
| Last full validation timestamp          | Most recent global validation point       |
| Active Extensions and annotation counts | Extension overlay inventory               |

Manifest item types:

| ITEM TYPE                  | FIELDS                                                                        | MEANING                                                                         |
|----------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| MISSING_MEDIATOR           | gpcl_node_id, message, rationale                                              | Logged when a GPCL-R6 target has no corresponding FCL behavioral mediator       |
| SUPERSEDE_FAILED           | source_node_id, attempted_replacement_content_hash, failure_reason, timestamp | Logged when supersession fails during replacement insert or rewiring            |
| SUPERSEDE_PENDING_DETECTED | node_id, prior_status, detected_at                                            | Logged by VERIFY when a node remains in SUPERSEDE_PENDING; severity is BLOCKING |

Semantic-gap classification:

| PROPERTY             | VALUE                                                                                                                 |
|----------------------|-----------------------------------------------------------------------------------------------------------------------|
| Allowed type(s)      | MISSING_MEDIATOR                                                                                                      |
| Required constraints | Must be logged explicitly; must carry human rationale; must be resolved or explicitly waived before system-wide CLEAN |

**Figure 7.2. Reconciliation manifest tracks and blocking item surfaces**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%%
erDiagram
accTitle: Reconciliation manifest tracks and blocking item surfaces
accDescr: Shows the reconciliation manifest, its tracked summary records, and the three typed pending-item families that affect CLEAN eligibility.
direction LR
RECONCILIATION_MANIFEST ||--|{ TIER_COUNT : tracks
RECONCILIATION_MANIFEST ||--|{ STATUS_DISTRIBUTION : summarizes
RECONCILIATION_MANIFEST ||--|{ EXTENSION_SUMMARY : counts
RECONCILIATION_MANIFEST ||--o{ MISSING_MEDIATOR : logs
RECONCILIATION_MANIFEST ||--o{ SUPERSEDE_FAILED : logs
RECONCILIATION_MANIFEST ||--o{ SUPERSEDE_PENDING_DETECTED : logs
RECONCILIATION_MANIFEST {
    datetime last_full_validation_timestamp
}
TIER_COUNT {
    string tier
    int total_nodes
}
STATUS_DISTRIBUTION {
    string status
    int count
}
EXTENSION_SUMMARY {
    int active_extensions
    int annotation_count
}
MISSING_MEDIATOR {
    string gpcl_node_id
    string message
    string rationale
}
SUPERSEDE_FAILED {
    string source_node_id
    string attempted_replacement_content_hash
    string failure_reason
    datetime timestamp
}
SUPERSEDE_PENDING_DETECTED {
    string node_id
    string prior_status
    datetime detected_at
    string severity
}
```

**Interpretation** The reconciliation manifest is not a generic notes bucket; it has a closed track structure and a small set of typed pending-item records that directly affect CLEAN eligibility.

**Authority basis** `operations.reconciliation_manifest_tracks`, `operations.reconciliation_manifest_schema`, and Section 7.3.

Illustrative manifest snippets:

```
pending_items:
- item_type: MISSING_MEDIATOR
  gpcl_node_id: GPCL-2.7
  message: "Latency target has no independent FCL behavioral mediator."
  rationale: "User-visible interaction breakdown has not been authored yet."

- item_type: SUPERSEDE_FAILED
  source_node_id: SAL-5.4
  attempted_replacement_content_hash: "sha256:0d8c..."
  failure_reason: "replacement_insert_validation_failed"
  timestamp: "2026-03-28T14:22:31Z"

- item_type: SUPERSEDE_PENDING_DETECTED
  node_id: CL-4.2
  prior_status: ACTIVE
  detected_at: "2026-03-28T14:24:00Z"
```

Interpret these three item families differently:

- `MISSING_MEDIATOR` is the only allowed semantic-gap classification in v6.3.
- `SUPERSEDE_FAILED` records an operational failure that must be diagnosed or retried.
- `SUPERSEDE_PENDING_DETECTED` is a blocking cleanliness failure, not a waivable semantic gap.

## 7.4 Compliance checklist

Structural validation requirements:

- All non-root nodes have `>= 1` valid, non-superseded parent citation.
- All parent citations reference the correct parent tier.
- No cycles exist in any citation path.
- Every node tier belongs to `active_tiers`.
- Every `system_definition` artifact includes at least one representative node for each active tier.
- No tier-skipping is present.
- All inline citations have matching `parent_ids`.
- No node remains `DIRTY`.
- No node remains `SUPERSEDE_PENDING`.
- The reconciliation manifest has zero pending items.
- Any declared semantic gap uses an allowed classification and is resolved or explicitly waived with rationale before `CLEAN`.
- If any Extension is active, all critical or blocking advisories have recorded disposition notes.

Atomic-rule validation requirements:

- Each tier satisfies its declared inclusion and exclusion rules.
- `FCL-R7` requires logical data-entity and CRUD enumeration for data-modifying capabilities.
- `GPCL-FCL-BR1` requires either an `FCL` mediator or a `MISSING_MEDIATOR` manifest entry.
- `CL` nodes remain declarative and must declare `constraint_origin`.
- `CL-R9` applies to `derived` constraints; `CL-R9-imposed` applies to `imposed` constraints.
- `CIT-R7` requires re-validation after cited parent version changes.
- `SAL` cites all active parent tiers.
- `ICL-R2` requires machine-parseable schemas.
- `ISL` stubs must cite `CDL` parent IDs.
- `CDL-R7` requires language-specific blueprints when multiple targets are declared.
- All `REVIEW_REQUIRED` items must carry recorded human disposition before affected nodes move from `DRAFT` to `ACTIVE`.

Extension validation requirements:

- Active Extensions declare compatible `DDR-Core-6.x` contracts.
- Extension annotations appear in `extension_annotations` only.
- Extension advisories are reviewed; non-critical advisories have disposition notes.
- ARE candidates are either promoted via `INSERT` or discarded.
- E5 declares a valid `scoring_profile`.
- Custom ARE profiles satisfy required fields and bounded, ordered, non-overlapping score bands.
- Candidates promoted below threshold require `override_flag: true` plus non-empty `human_rationale`.

## 7.5 CLEAN-state logic

A DDR graph may be treated as `CLEAN` only when all of the following are true:

- No node is `DIRTY`.
- No node is `SUPERSEDE_PENDING`.

- Citation rules and topology invariants hold.
- Tier-specific atomic rules hold.
- Reconciliation manifest pending items are resolved or explicitly waived where allowed.
- Parent-version freshness is preserved.
- Extension advisories with critical or blocking severity have disposition.

**Figure 7.3. CLEAN-state validation gate**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart TD
accTitle: CLEAN-state validation gate accDescr: Shows the ordered checks that must all pass before a DDR graph may be declared CLEAN. START["Candidate CLEAN assertion"] --> STATUS["Any DIRTY or SUPERSEDE_PENDING nodes?"] STATUS -->|yes| FAIL["Not CLEAN"] STATUS -->|no| TOPO["Topology and citation rules hold?"] TOPO -->|no| FAIL TOPO -->|yes| ATOMIC["Tier atomic rules and review dispositions complete?"] ATOMIC -->|no| FAIL ATOMIC -->|yes| MANIFEST["Manifest pending items resolved or waived where allowed?"] MANIFEST -->|no| FAIL MANIFEST -->|yes| EXT["Critical or blocking extension advisories disposed?"] EXT -->|no| FAIL EXT -->|yes| CLEAN["CLEAN"] classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px; classDef alert fill:#ffebee,stroke:#c62828,color:#b71c1c,stroke-width:1.5px; class START,TOPO,ATOMIC,MANIFEST,EXT,CLEAN normative; class STATUS,FAIL alert;
```

**Interpretation** CLEAN is a gate condition across status, structure, atomic-rule review, manifest state, and extension-advisory disposition; no single operation can declare it unilaterally.

**Authority basis** `compliance_checklist`, `operations.semantic_consistency_rules`, and Section 7.5.

See also: Section 5, Section 9, Section 10.

## 8. Extension System and ARE

This section covers the optional extension overlay model, integration rules, ARE candidate-pool behavior, scoring profiles, and the v6.3 extension catalog. Use Section 9 for the schema-side extension and ARE contract and Section 10 for crosswalk and historical lookup surfaces.

### 8.1 Extension architecture

**Figure 8.1. Extension overlay architecture**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: Extension overlay architecture accDescr: Shows Core DDR surfaces, the extension overlay, manifest advisories, and the ARE candidate pool without using v11-only architecture-beta syntax.
subgraph CORE["Core DDR surfaces"] CORENODE["Core DAG"] ANN["extension_annotations"] end
subgraph EXT["Extension overlay"] EXTRT["Extension runtime"] end
subgraph AUX["Auxiliary surfaces"] MANIFEST["Reconciliation manifest advisories"] POOL["ARE candidate pool"]
INSERT["Human-reviewed INSERT into Core"] end
EXTRT -->|reads| CORENODE
EXTRT -->|writes namespaced annotations| ANN
EXTRT -->|emits advisories| MANIFEST
EXTRT -->|ARE only| POOL
POOL -->|promotion via INSERT only| INSERT
INSERT --> CORENODE
classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px;
classDef schema fill:rgba(59,130,246,0.1),stroke:rgba(59,130,246,0.5),color:#60a5fa,stroke-width:2px;
classDef extension fill:#f3e5f5,stroke:#6a1b9a,color:#4a148c,stroke-width:1.5px;
class CORENODE,MANIFEST,INSERT normative;
class ANN schema;
class EXTRT,POOL extension;
```

**Interpretation** Extensions remain outside the Core semantics boundary: they may read, annotate, advise, and stage candidates, but only Core operations can mutate the authoritative DAG.

**Authority basis** `extension_system.architecture_description`, `permitted_actions`, `candidate_pool`, and Section 8.1.

Permitted actions:

- Read Core node content.
- Annotate Core nodes with namespaced metadata stored only in `extensions_annotations`.
- Generate derived external artifacts such as reports, IaC, or recommendations.
- Add advisories to the reconciliation manifest's extension-advisories surface.

Prohibited actions:

- Modify any Core node's `content`, `parent_ids`, `tier`, or `status`.
- Redefine Core tier semantics or atomic rules.
- Introduce structural cycles into the Core DAG.
- Set Core nodes to `DIRTY`; extensions may advise but not mutate status.

Normative note:



- "All Core tiers" is not a valid contract declaration. Extensions must enumerate tiers by name to satisfy EXT-R2.

## 8.2 Extension integration rules

| RULE   | STATEMENT                                                                                        |
|--------|--------------------------------------------------------------------------------------------------|
| EXT-R1 | Must declare contract version compatible with <code>DDR-Core-6.x</code> .                        |
| EXT-R2 | Must declare which Core tiers the extension reads and annotates.                                 |
| EXT-R3 | Annotations must be namespaced by Extension ID, such as <code>HRE::min_hardware_profile</code> . |
| EXT-R4 | Extensions update the reconciliation manifest; annotation counts are tracked.                    |
| EXT-R5 | Disabling an Extension leaves Core CLEAN and DIRTY status unchanged.                             |
| EXT-R6 | Extension-internal derived artifact graphs must maintain their own acyclicity.                   |
| EXT-R7 | Extension advisories do not mutate Core node status.                                             |

**Figure 8.2. Extension integration boundary map**

```

%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor':
'#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a',
'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize':
'14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: Extension integration boundary map accDescr: Separates the narrow allowed extension
behaviors from the prohibited mutation and semantic redefinition surfaces. EXT["Extension
runtime"] subgraph ALLOWED["Allowed behaviors"] READ["Read Core node content"]
ANN["Annotate via extension_annotations"] ADV["Advisories to reconciliation manifest"] end
subgraph BLOCKED["Blocked behaviors"] MUT["No content, parent_ids, tier, or status mutation"]
REDEF["No redefinition of Core semantics"] CYCLE["No structural cycles introduced"] end EXT -->
READ EXT --> ANN EXT --> ADV EXT -. forbidden .-> MUT EXT -. forbidden .-> REDEF EXT -.
forbidden .-> CYCLE classDef normative
fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px; classDef
extension fill:#f3e5f5,stroke:#6a1b9a,color:#4a148c,stroke-width:1.5px; classDef alert
fill:#ffebee,stroke:#c62828,color:#b71c1c,stroke-width:1.5px; class EXT extension; class
READ,ANN,ADV normative; class MUT,REDEF,CYCLE alert;

```

**Interpretation** The extension contract is intentionally asymmetric: allowed behaviors are narrow and explicit, while prohibited behaviors ring-fence the Core from hidden mutation.

**Authority basis** `extension_system.permitted_actions`, `prohibited_actions`, and EXT-R1 through EXT-R7.

## 8.3 ARE candidate pool and activation states

The candidate pool is specific to E5, the AI Upward Reconstruction Engine ( `ARE` ). It is explicitly outside the Core DAG until promotion through `INSERT` .

| PROPERTY               | VALUE                                                                                                           |
|------------------------|-----------------------------------------------------------------------------------------------------------------|
| Candidate status value | <code>CANDIDATE</code> (not a Core status)                                                                      |
| Visibility rule        | Visible when <code>ARE</code> is <code>active</code> or <code>paused</code> ; hidden when <code>disabled</code> |
| Checkpoint path        | <code>.agent/state/are_candidate_pool.checkpoint.yaml</code>                                                    |
| Effect on Core status  | No effect on Core <code>CLEAN</code> or <code>DIRTY</code> status                                               |
| Promotion mechanism    | Promotion into Core requires <code>INSERT</code> with full validation and threshold checks                      |
| Discard trigger        | Any transition to <code>disabled</code> discards the pool and deletes the checkpoint file                       |

Activation states:

| STATE                 | INFERENCE            | POOL VISIBILITY  | POOL PRESERVED AT RUNTIME | POOL PRESERVED ACROSS RESTART |
|-----------------------|----------------------|------------------|---------------------------|-------------------------------|
| <code>active</code>   | <code>running</code> | <code>yes</code> | <code>yes</code>          | <code>optional</code>         |
| <code>paused</code>   | <code>halted</code>  | <code>yes</code> | <code>yes</code>          | <code>yes</code>              |
| <code>disabled</code> | <code>halted</code>  | <code>no</code>  | <code>no</code>           | <code>no</code>               |

Activation transitions:

| FROM     | TO       | PERMITTED | EFFECT                                                                 |
|----------|----------|-----------|------------------------------------------------------------------------|
| active   | paused   | yes       | Inference halts; pool is atomically checkpointed and remains browsable |
| paused   | active   | yes       | Inference resumes; pool, scores, and annotations are retained          |
| paused   | disabled | yes       | Checkpoint file is deleted and pool is discarded                       |
| active   | disabled | yes       | Pool is discarded                                                      |
| disabled | active   | yes       | ARE starts fresh with an empty pool                                    |
| disabled | paused   | no        | Forbidden because no candidate pool exists in disabled state           |

Operational reading of the activation contract:

- `pool_preserved_runtime` and `pool_preserved_restart` are first-class ARE state semantics, not informal implementation hints.
- In `active`, restart preservation is explicitly `optional`; in `paused`, restart preservation is mandatory.
- Entering `paused` creates a durability obligation: the checkpoint must be written atomically on entry and re-written after every mutating pool action while paused.
- Transitioning to `disabled` deletes the checkpoint and discards the pool regardless of whether the prior state was `active` or `paused`.

Paused-state practitioner sequence:

1. ARE runs in `active` and accumulates candidates.
2. Transition `active` -> `paused` halts inference and writes `.agent/state/are_candidate_pool.checkpoint.yaml`.
3. Promotion by `INSERT` or manual discard remains allowed while paused.
4. Every paused-state pool mutation re-persists the checkpoint.
5. Process restart restores the checkpoint automatically and returns ARE to `paused`.
6. Any transition to `disabled` deletes the checkpoint and discards the pool.

## 8.4 ARE scoring profiles

Profile summary:

| PROFILE         | THRESHOLD | NOTES                                                     |
|-----------------|-----------|-----------------------------------------------------------|
| standard_v1     | 0.35      | Default E5 profile in the authoritative extension catalog |
| conservative_v1 | 0.55      | Intended for regulated or high-assurance                  |

|        |                 |                                                         |
|--------|-----------------|---------------------------------------------------------|
|        |                 | environments                                            |
| custom | template-driven | Must satisfy all required fields and conformance checks |

Shared standard and conservative input signals:

| SIGNAL ID                  | MEANING                                                                                   | WEIGHT CATEGORY |
|----------------------------|-------------------------------------------------------------------------------------------|-----------------|
| direct_source_node_count   | Counts directly cited source nodes supporting the candidate inference                     | high            |
| cross_tier_convergence     | Measures whether evidence converges across adjacent DDR tiers for the same inferred claim | high            |
| icl_contract_corroboration | Checks whether ICL contract definitions corroborate candidate semantics                   | medium          |
| sal_pattern_alignment      | Evaluates alignment with declared SAL architectural patterns                              | medium          |
| tier_diversity_index       | Assesses how many distinct eligible source tiers contribute evidence                      | low             |

standard\_v1 score bands:

| BAND            | RANGE     | GUIDANCE                                                                                           |
|-----------------|-----------|----------------------------------------------------------------------------------------------------|
| speculative     | 0.0 - 0.4 | Weak evidence; requires substantial human scrutiny before promotion consideration                  |
| probable        | 0.4 - 0.7 | Moderate evidence; promotion consideration is allowed only when human review confirms traceability |
| high_confidence | 0.7 - 1.0 | Strong evidence; prioritize for review and possible promotion via INSERT                           |

conservative\_v1 score bands:

| BAND | RANGE | GUIDANCE |
|------|-------|----------|
|      |       |          |

|                 |           |                                                                                                          |
|-----------------|-----------|----------------------------------------------------------------------------------------------------------|
| speculative     | 0.0 - 0.4 | Do not promote under normal conditions; require explicit documented justification for exception handling |
| probable        | 0.4 - 0.7 | Permit review only with heightened scrutiny and complete evidence traceability                           |
| high_confidence | 0.7 - 1.0 | Preferred band for promotion decisions after formal reviewer confirmation                                |

Override policy for both standard and conservative profiles:

- A candidate below `minimum_surfacing_threshold` may enter review only when it carries `override_flag: true` and a non-empty `human_rationale`.

Custom-profile contract:

| REQUIREMENT     | VALUE                                                                                                                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Required fields | <code>profile_id</code> , <code>input_signals</code> , <code>signal</code> subfields, <code>score_bands</code> , <code>band</code> subfields, <code>minimum_surfacing_threshold</code> , <code>override_policy</code> |
| Template rule   | Custom profiles may change field values and add signals or bands, but the object structure must remain conformant                                                                                                     |
| Validation note | Custom profiles fail extension contract validation when required fields are missing; deterministic ARE conformance validation also checks reference resolution, score-band ordering, and non-overlap                  |

Important reading of `are_scoring_profiles`:

- `standard_v1` and `conservative_v1` are concrete reusable profiles.
- `custom` is not a ready-made runtime profile. It is the required-fields contract plus the template shape that any implementation-defined custom profile must satisfy.
- Threshold gating controls whether low-confidence candidates are surfaced for review, not whether the model is allowed to infer candidates internally.

Example promotion-readiness comparison:

| CANDIDATE SCORE | STANDARD_V1 (0.35) | CONSERVATIVE_V1 (0.55) | REVIEW POSTURE                    |
|-----------------|--------------------|------------------------|-----------------------------------|
| 0.62            | Above threshold    | Above threshold        | Reviewable without override under |

|      |                 |                 |                                                                        |
|------|-----------------|-----------------|------------------------------------------------------------------------|
|      |                 |                 | both concrete profiles                                                 |
| 0.40 | Above threshold | Below threshold | Reviewable under standard_v1; requires override under conservative_v1  |
| 0.30 | Below threshold | Below threshold | Requires override_flag: true plus non-empty human_rationale under both |

Minimal E5 contract example:

```
id: E5
name: AI Upward Reconstruction Engine (ARE)
contract: ARE-1.0 / DDR-Core-6.x
scoring_profile: standard_v1
reads: [ISL, CDL, ICL, SAL]
annotates: [SAL, ICL, CDL, ISL]
```

**Figure 8.3. ARE candidate-pool and scoring lifecycle**

```
%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor': '#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a', 'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize': '14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%%
flowchart TB
    accTitle: ARE candidate-pool and scoring lifecycle
    accDescr: Separates the ARE activation state machine from the candidate scoring and promotion pipeline, including checkpointing and the forbidden disabled-to-paused transition.
    subgraph ACT ["ARE activation lifecycle"]
        ACTIVE["active"]
        PAUSED["paused"]
        DISABLED["disabled"]
        CHECKPOINT["Write or reload checkpoint"]
        DROP["Delete checkpoint and discard pool"]
        FORBID["disabled -> paused is forbidden"]
        ACTIVE --> PAUSED
        PAUSED --> ACTIVE
        ACTIVE --> DISABLED
        PAUSED --> DISABLED
        PAUSED --> CHECKPOINT
        DISABLED --> DROP
        DISABLED -. no candidate pool exists .-> FORBID
    end
    subgraph PIPE ["Candidate scoring and promotion pipeline"]
        INFER["Infer candidate"]
        POOL["Candidate Pool"]
        SCORE["Score under declared profile"]
        THRESH["Meets threshold or valid override?"]
        REVIEW["Human review"]
        INSERT["Promote via INSERT into Core"]
        HOLD["Remain candidate or discard"]
        INFER --> POOL
        POOL --> SCORE
        SCORE --> THRESH
        THRESH --> |yes| REVIEW
        REVIEW --> INSERT
        THRESH --> |no| HOLD
    end
    ACTIVE --> INFER
    CHECKPOINT -. preserves pool for restart .-> POOL
    classDef normative fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px;
    classDef caution fill:rgba(245,158,11,0.1),stroke:rgba(245,158,11,0.5),color:#fbbf24,stroke-width:2px;
    classDef alert fill:#ffebee,stroke:#c62828,color:#b71c1c,stroke-width:1.5px;
    classDef extension fill:#f3e5f5,stroke:#6a1b9a,color:#4a148c,stroke-width:1.5px;
    class ACTIVE,PAUSED,CHECKPOINT,INFER,POOL,SCORE extension;
    class THRESH,HOLD caution;
    class REVIEW,INSERT normative;
    class DISABLED,DROP,FORBID alert;
```

**Interpretation** ARE inference, scoring, checkpointing, and promotion are coupled but not identical: candidate generation is extension-local, while promotion remains a human-reviewed Core INSERT .

**Authority basis** extension\_system.candidate\_pool , are\_scoring\_profiles , the E5 catalog entry, and Sections 8.3-8.4 .

8.5 Extension catalog

E1 - Hardware & Resource Intelligence Extension ( HRE )

| PROPERTY  | VALUE                  |  |
|-----------|------------------------|--|
| Contract  | HRE-1.0 / DDR-Core-6.x |  |
| Reads     | CL, SAL, CDL, ISL      |  |
| Annotates | CL, SAL                |  |

| RULE   | STATEMENT                                                                             |
|--------|---------------------------------------------------------------------------------------|
| HRE-R1 | Bottom-up inference produces minimum hardware profiles as CL-compatible declarations. |
| HRE-R2 | Cloud recommendations include at least two provider-agnostic instance class options.  |
| HRE-R3 | Top-down enforcement validates that SAL patterns do not exceed CL ceilings.           |
| HRE-R4 | All recommendations are advisory and do not override CL without explicit MODIFY.      |

E2 - Dependency Graph Analyzer ( DGA )

| PROPERTY  | VALUE                  |  |
|-----------|------------------------|--|
| Contract  | DGA-1.0 / DDR-Core-6.x |  |
| Reads     | CL, ICL, CDL, ISL      |  |
| Annotates | CL, ICL                |  |

| RULE   | STATEMENT                                                                    |
|--------|------------------------------------------------------------------------------|
| DGA-R1 | Produces a complete directed dependency graph for all CL-declared libraries. |

|        |                                                                                     |
|--------|-------------------------------------------------------------------------------------|
| DGA-R2 | Detects version conflicts and suggests resolutions.                                 |
| DGA-R3 | Transitive dependency reports flag copyleft licenses that could impose constraints. |

### E3 - Lifecycle & Versioning Engine ( LVE )

| PROPERTY  | VALUE                                       |  |
|-----------|---------------------------------------------|--|
| Contract  | LVE-1.0 / DDR-Core-6.x                      |  |
| Reads     | XPD, SIL, GPCL, FCL, CL, SAL, ICL, CDL, ISL |  |
| Annotates | XPD, SIL, GPCL, FCL, CL, SAL, ICL, CDL, ISL |  |

| RULE   | STATEMENT                                                                                       |
|--------|-------------------------------------------------------------------------------------------------|
| LVE-R1 | Every node modification produces a version-history entry with timestamp, author, and rationale. |
| LVE-R2 | Technical debt items are classified by tier origin and estimated remediation effort.            |
| LVE-R3 | Deprecation requires a sunset date and migration path before node status becomes DEPRECATED.    |
| LVE-R4 | Version-control integration maps DDR node IDs to VCS commit hashes.                             |

### E4 - Observability & Runtime Engine ( ORE )

| PROPERTY  | VALUE                    |  |
|-----------|--------------------------|--|
| Contract  | ORE-1.0 / DDR-Core-6.x   |  |
| Reads     | GPCL, SAL, ICL, CDL, ISL |  |
| Annotates | ISL, SAL                 |  |

| RULE   | STATEMENT                                                             |
|--------|-----------------------------------------------------------------------|
| ORE-R1 | Telemetry stubs are derived from GPCL latency and throughput targets. |



|        |                                                                                                                             |
|--------|-----------------------------------------------------------------------------------------------------------------------------|
| ORE-R2 | Alert rules are expressed in vendor-agnostic format.                                                                        |
| ORE-R3 | Every <code>SAL</code> component must have at least one telemetry point for operational readiness.                          |
| ORE-R4 | Incident-to-design traceability maps runtime anomalies to <code>ISL</code> , <code>CDL</code> , and <code>SAL</code> nodes. |

## E5 - AI Upward Reconstruction Engine ( ARE )

| PROPERTY        | VALUE                                                                                                                                                                                                                       |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Contract        | ARE-1.0 / DDR-Core-6.x                                                                                                                                                                                                      |
| Scoring profile | <code>standard_v1</code>                                                                                                                                                                                                    |
| Reads           | <code>ISL</code> , <code>CDL</code> , <code>ICL</code> , <code>SAL</code>                                                                                                                                                   |
| Annotates       | <code>SAL</code> , <code>ICL</code> , <code>CDL</code> , <code>ISL</code>                                                                                                                                                   |
| Notes           | ARE annotation is restricted to <code>SAL</code> , <code>ICL</code> , <code>CDL</code> , and <code>ISL</code> . Higher-level intent, governance, ethical, or functional insights are surfaced only as candidate-pool items. |

| RULE   | STATEMENT                                                                                                                                                                                                                                                                                                                                    |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ARE-R1 | All inferred nodes are placed in the extension candidate pool; automatic promotion is prohibited.                                                                                                                                                                                                                                            |
| ARE-R2 | Every candidate carries <code>ARE::confidence_score</code> computed under the declared <code>scoring_profile</code> ; deterministic conformance validation resolves the profile and enforces reproducible scoring.                                                                                                                           |
| ARE-R3 | Promotion into the Core DAG requires <code>INSERT</code> with full atomic validation.                                                                                                                                                                                                                                                        |
| ARE-R4 | ARE must never autonomously create <code>XPD</code> or <code>GPCL</code> nodes.                                                                                                                                                                                                                                                              |
| ARE-R5 | Every ARE deployment must declare a <code>scoring_profile</code> that references a profile defined in <code>are_scoring_profiles</code> .                                                                                                                                                                                                    |
| ARE-R6 | ARE must implement the tri-state activation lifecycle <code>active</code> , <code>paused</code> , and <code>disabled</code> , with <code>disabled -&gt; paused</code> forbidden.                                                                                                                                                             |
| ARE-R7 | On every <code>active -&gt; paused</code> transition, ARE must atomically persist the full candidate pool to <code>.agent/state/are_candidate_pool.checkpoint.yaml</code> , re-persist after each mutating pool action while paused, restore paused state on restart, and delete the checkpoint on any transition to <code>disabled</code> . |

#### E6 - Security & Compliance Engine ( SCE )

| PROPERTY  | VALUE                  |  |
|-----------|------------------------|--|
| Contract  | SCE-1.0 / DDR-Core-6.x |  |
| Reads     | GPCL, CL, SAL, ICL     |  |
| Annotates | GPCL, SAL, ICL         |  |

| RULE   | STATEMENT                                                                       |
|--------|---------------------------------------------------------------------------------|
| SCE-R1 | Threat models are expressed in STRIDE format or equivalent structured notation. |
| SCE-R2 | Trust-boundary violations in SAL are flagged as high-priority advisories.       |
| SCE-R3 | Every ICL contract must have an explicit RBAC access-control policy.            |
| SCE-R4 | PII data flows in ICL must be traceable to GPCL data-residency constraints.     |
| SCE-R5 | Compliance evidence records are immutable once generated.                       |

#### E7 - Data Domain Extension ( DDE )

| PROPERTY  | VALUE                                                                                                                 |
|-----------|-----------------------------------------------------------------------------------------------------------------------|
| Contract  | DDE-1.0 / DDR-Core-6.x                                                                                                |
| Reads     | FCL, GPCL, SAL, ICL, CDL                                                                                              |
| Annotates | ICL, SAL, FCL                                                                                                         |
| Notes     | When annotating FCL, DDE performs confirmation-only validation. It does not infer missing data entities for the Core. |

| RULE   | STATEMENT                                                                      |
|--------|--------------------------------------------------------------------------------|
| DDE-R1 | Canonical ER models are expressed in formal notation such as ERD or DBML.      |
| DDE-R2 | Every ICL payload schema is validated against the canonical ER model.          |
| DDE-R3 | Schema-consistency violations are flagged as blocking advisories.              |
| DDE-R4 | Data lifecycle policies specify retention periods traceable to GPCL regulatory |

|        |                                                                                                                                                                                                                                                    |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|        | requirements.                                                                                                                                                                                                                                      |
| DDE-R5 | When annotating <code>FCL</code> , DDE verifies only that each entity named under <code>FCL-R7</code> has a corresponding <code>ICL</code> schema. Missing <code>FCL-R7</code> enumeration is a Core validation failure, not a DDE discovery task. |

#### E8 - Deployment & CI/CD Planner ( DCP )

| PROPERTY  | VALUE                  |  |
|-----------|------------------------|--|
| Contract  | DCP-1.0 / DDR-Core-6.x |  |
| Reads     | CL, SAL, ISL           |  |
| Annotates | ISL, SAL               |  |

| RULE   | STATEMENT                                                                               |
|--------|-----------------------------------------------------------------------------------------|
| DCP-R1 | Deployment manifests map every <code>SAL</code> subsystem to a deployment unit.         |
| DCP-R2 | CI/CD pipeline definitions include at least lint, test, build, and deploy stages.       |
| DCP-R3 | All generated IaC cites the <code>CL</code> nodes from which configuration was derived. |
| DCP-R4 | Environment-specific configuration is separated from application code.                  |

#### E9 - Ethics & Human-Centered Design Extension ( EHD )

| PROPERTY  | VALUE                   |  |
|-----------|-------------------------|--|
| Contract  | EHD-1.0 / DDR-Core-6.x  |  |
| Reads     | XPD, SIL, FCL, SAL, CDL |  |
| Annotates | FCL, CDL, SAL           |  |

| RULE   | STATEMENT                                                                                                     |
|--------|---------------------------------------------------------------------------------------------------------------|
| EHD-R1 | Bias-impact assessments identify affected demographic groups and potential algorithmic biases.                |
| EHD-R2 | Accessibility compliance validates <code>FCL</code> capabilities against WCAG 2.1 AA or a <code>GPCL</code> - |

|        |                                                                                                                                                                                                                                                                                                    |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|        | declared standard.                                                                                                                                                                                                                                                                                 |
| EHD-R3 | Algorithmic accountability maps link each automated <code>CDL</code> decision to a human oversight mechanism.                                                                                                                                                                                      |
| EHD-R4 | All EHD assessments cite the <code>XPD</code> ethical boundary conditions being evaluated.                                                                                                                                                                                                         |
| EHD-R5 | When <code>XPD</code> is inactive, EHD creates a synthetic <code>XPD</code> -equivalent risk-flagging artifact anchored to <code>SIL</code> ; it carries no precedence weight, cannot be cited in Core <code>parent_ids</code> , and cannot substitute for a human-authored <code>XPD</code> node. |

See also: Section 9, Section 10.

## 9. Schema Contract and Machine Validation Surface

This section covers the machine-contract side of DDR v6.3: profile branching, canonical top-level closures, node and citation schemas, express-specific rules, extension and ARE schema rules, and lifecycle schema requirements. Use Section 3 for the human-readable structural model, Section 5 for lifecycle behavior, Section 6 for Express Mode semantics, and Section 8 for extension semantics.

### 9.1 Top-level contract by profile

Global top-level schema rules:

- `type: object`
- `additionalProperties: false`
- Always required: `ddr_version`, `document_profile`, `active_tiers`, `nodes`

Profile-specific requirements:

| PROFILE                               | REQUIRED / PROHIBITED BEHAVIOR                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>project_instance</code>         | Must not require <code>system_metadata</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <code>project_instance_express</code> | Requires <code>express_mode</code> ; each node must require <code>express_mode_group</code> ; must not require <code>system_metadata</code>                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <code>system_definition</code>        | Requires <code>system_metadata</code> , <code>axioms</code> , <code>edge_type_definitions</code> , <code>node_schema_fields</code> , <code>node_id_format</code> , <code>dag_invariants</code> , <code>citation_rules</code> , <code>consumption_modes</code> , <code>express_mode</code> , <code>tier_definitions</code> , <code>constraint_precedence</code> , <code>operations</code> , <code>extension_system</code> , <code>extension_catalog</code> , <code>compliance_checklist</code> , <code>glossary</code> , <code>are_scoring_profiles</code> , and <code>lifecycle</code> |

Top-level conditional branches enforced by the schema `allOf` surface:

| CONDITION                                                | ENFORCED CONSEQUENCE                                                                                                                       |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>document_profile = project_instance</code>         | <code>system_metadata</code> must not be required                                                                                          |
| <code>document_profile = project_instance_express</code> | <code>express_mode</code> required; each node requires <code>express_mode_group</code> ; <code>system_metadata</code> must not be required |
|                                                          |                                                                                                                                            |

|                                                                             |                                                        |
|-----------------------------------------------------------------------------|--------------------------------------------------------|
| document_profile = system_definition                                        | Full authoritative top-level surface required          |
| active_tiers contains XPD                                                   | Any SIL node must have at least one parent citation    |
| project.mode = express                                                      | document_profile is forced to project_instance_express |
| document_profile =<br>project_instance_express plus<br>project.mode present | project.mode must equal express                        |

**Figure 9.1. Profile root and schema-closure relationship**

```

%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor':
'#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a',
'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize':
'14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% flowchart LR
accTitle: Profile root and schema-closure relationship accDescr: Shows the shared root contract
and the profile-specific closure rules enforced by the schema for project, express, and system-
definition artifacts. subgraph SHARED["Shared root contract"] ROOT["Schema root"] REQ["Always
required: ddr_version, document_profile, active_tiers, nodes"] NODES["nodes[] -> DdrNode"] end
subgraph PROFILE["Profile-specific closure"] PI["project_instance"] PIE["project_instance_express"]
SD["system_definition"] EM["Requires express_mode and node express_mode_group"]
FULL["Requires full authoritative top-level surface"] end ROOT --> REQ --> NODES ROOT --> PI
ROOT --> PIE --> EM ROOT --> SD --> FULL classDef normative
fill:rgba(34,197,94,0.1),stroke:rgba(34,197,94,0.5),color:#4ade80,stroke-width:2px; classDef schema
fill:rgba(59,130,246,0.1),stroke:rgba(59,130,246,0.5),color:#60a5fa,stroke-width:2px; class
ROOT,REQ,NODES,PI,PIE,SD schema; class EM,FULL normative;

```

**Interpretation** The schema root is both closed and profile-aware: every valid artifact shares the same minimal root keys, then branches into express-specific or system-definition-specific obligations.

**Authority basis** `ddr_node_schema_v6.3.yaml` root properties, profile branching, and Section 9.1.

## 9.2 active\_tiers schema closure

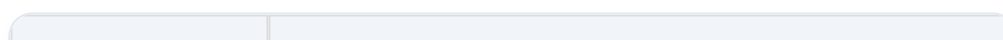
The schema enforces:

- array type
- unique tier values
- values restricted to the canonical tier enum
- exactly one of the four ordered topologies listed in Section 3.3

This means DDR v6.3 does not permit arbitrary tier activation orderings.

## 9.3 DdrNode definition

The schema's `DdrNode` contract enforces:



| AREA                               | MACHINE RULE                                                                                                                                                        |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Required fields                    | <code>id</code> , <code>tier</code> , <code>title</code> , <code>status</code> , <code>version</code> , <code>created</code> , <code>modified</code>                |
| Status enum                        | <code>DRAFT</code> , <code>ACTIVE</code> , <code>DIRTY</code> , <code>DEPRECATED</code> , <code>SUPERSEDED</code> , <code>SUPERSEDE_PENDING</code>                  |
| ID pattern                         | General regex permits <code>XPD-0.N</code> or <code>[A-Z]{2,5}-N.M</code> ; tier-specific branches tighten each tier's allowed prefix                               |
| Root handling                      | <code>XPD</code> may be root when active; <code>SIL</code> may be root when <code>XPD</code> is inactive; other non-root tiers require at least one parent citation |
| <code>constraint_origin</code>     | Allowed only when <code>tier = CL</code> ; forbidden for other tiers                                                                                                |
| <code>prior_status</code>          | Required when <code>status = SUPERSEDE_PENDING</code> ; forbidden otherwise                                                                                         |
| <code>express_mode_group</code>    | Allowed values <code>G1-G4</code> ; required for express-profile project artifacts                                                                                  |
| <code>extension_annotations</code> | Must be namespaced as <code>EXTENSION_ID::annotation_key</code> ; reserved Core field names cannot appear after <code>::</code>                                     |
| Additional properties              | Forbidden                                                                                                                                                           |

#### 9.4 ParentCitation definition

| PROPERTY                     | MACHINE RULE                                                                                                              |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Required fields              | <code>id</code> , <code>edge_type</code>                                                                                  |
| Allowed edge types           | <code>derives</code> , <code>constrains</code> , <code>implements</code>                                                  |
| Forbidden Core parent edge   | <code>extends</code> is not allowed in <code>parent_ids</code>                                                            |
| <code>derivation_mode</code> | Allowed values are <code>semantic</code> and <code>traceability</code> ; valid only when <code>edge_type = derives</code> |
| Additional properties        | Forbidden                                                                                                                 |

#### 9.5 Express-specific schema rules

| AREA                          | MACHINE RULE                                                                                                                                                            |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ExpressModeGroup</code> | Each group ID is closed to its exact tier tuple: <code>G1 = XPD, SIL, GPCL</code> ; <code>G2 = FCL, CL</code> ; <code>G3 = SAL, ICL</code> ; <code>G4 = CDL, ISL</code> |

|                                               |                                                                                                                   |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Top-level<br><code>express_mode.groups</code> | Must contain all four groups                                                                                      |
| Express-profile nodes                         | Every node requires <code>express_mode_group</code> when <code>document_profile = project_instance_express</code> |
| Project mode note                             | If <code>project.mode</code> is present for an express-profile artifact, it must be <code>express</code>          |

## 9.6 Extension and ARE schema rules

| AREA                                 | MACHINE RULE                                                                                                                                                                                                                                                  |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ExtensionEntry</code>          | Requires <code>id</code> , <code>name</code> , <code>contract</code> , <code>reads</code> , <code>annotates</code> , and <code>rules</code> ; <code>notes</code> is optional; <code>additionalProperties: false</code>                                        |
| E5 special case                      | When <code>id = E5</code> , <code>scoring_profile</code> is required                                                                                                                                                                                          |
| <code>AreActivationState</code>      | Requires <code>inference</code> , <code>pool_visibility</code> , <code>pool_preserved_runtime</code> , <code>pool_preserved_restart</code> , <code>promotion_allowed</code> , and <code>discard_allowed</code> ; <code>checkpoint_behavior</code> is optional |
| <code>AreActivationTransition</code> | Requires <code>from</code> and <code>to</code> ; if <code>permitted = false</code> , <code>rationale</code> is required and <code>effect</code> is forbidden; otherwise <code>effect</code> is required                                                       |
| <code>AreActivationStates</code>     | Requires <code>active</code> , <code>paused</code> , <code>disabled</code> , and <code>transitions</code>                                                                                                                                                     |
| <code>ScoringProfile</code>          | Requires <code>input_signals</code> , <code>score_bands</code> , <code>minimum_surfacing_threshold</code> , and <code>override_policy</code>                                                                                                                  |
| Score bands                          | Each band must provide <code>band_id</code> , two-number <code>range</code> , <code>label</code> , and <code>promotion_guidance</code>                                                                                                                        |
| Custom profile conformance           | Custom profiles must satisfy <code>required_fields</code> and the <code>profile_template</code> shape; deterministic ARE validation additionally checks reference resolution, band ordering, and non-overlap                                                  |
| Numeric bounds                       | All score values are constrained to <code>[0,1]</code>                                                                                                                                                                                                        |

## 9.7 Lifecycle schema rules

| AREA                   | MACHINE RULE                                                                                                                                                |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>lifecycle</code> | Object with <code>additionalProperties: false</code> ; requires <code>status_transitions</code> ; <code>guard_definitions</code> is optional but typed when |



|                            |                                                                                                                                 |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------|
|                            | present                                                                                                                         |
| StatusTransition core rule | Always requires from and operation; requires to unless phase = rollback, in which case it requires to_node_field = prior_status |
| phase                      | Closed enum: commit or rollback                                                                                                 |
| side_effect                | Closed enum: propagation                                                                                                        |
| prerequisite_operations    | Optional unique array whose values must be canonical operation names                                                            |
| guards                     | Optional array closed to gc-001 through gc-009 via GuardIdRef                                                                   |
| GuardDefinition            | Requires id, description, and verification_mode                                                                                 |
| Guard verification_mode    | Closed enum: structural or manual                                                                                               |

## 9.8 Schema Conditionals, Enums, and Rule-ID Typing

Closed enum and reference surfaces worth keeping at hand:

| SURFACE                                 | ALLOWED VALUES OR SHAPE                                                                                             |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| StatusEnum                              | DRAFT, ACTIVE, DIRTY, DEPRECATED, SUPERSEDED, SUPERSEDE_PENDING                                                     |
| OperationNameEnum                       | INSERT, DELETE, MODIFY, SUPERSEDE, VERIFY, VALIDATE, UNBUNDLE_SCAN, UNBUNDLE_EXECUTE                                |
| TransitionPhase                         | commit, rollback                                                                                                    |
| TransitionSideEffect                    | propagation                                                                                                         |
| GuardIdRef                              | gc-001 through gc-009                                                                                               |
| ManifestItemType.item_type              | MISSING_MEDIATOR, SUPERSEDE_FAILED, SUPERSEDE_PENDING_DETECTED                                                      |
| SemanticGapClassification.allowed_types | MISSING_MEDIATOR only                                                                                               |
| extension_annotations key shape         | EXTENSION_ID::annotation_key, with reserved Core suffixes such as content, parent_ids, status, tier, and id blocked |

Rule-ID families are also typed rather than free-form:

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

| FAMILY           | SCHEMA PATTERN                         | EXAMPLE      |
|------------------|----------------------------------------|--------------|
| InvariantId      | <code>^INV-[0-9]+\$</code>             | INV-7        |
| CitationRuleId   | <code>^CIT-R[0-9]+\$</code>            | CIT-R6       |
| AtomicTierRuleId | <code>^(?:XPD</code>                   | SIL          |
| BridgeRuleId     | <code>^[A-Z]+-[A-Z]+-BR[0-9]+\$</code> | GPCL-FCL-BR1 |
| ExtensionRuleId  | <code>^[A-Z]{3,4}-R[0-9]+\$</code>     | ARE-R7       |

**Figure 9.2. Schema definition map for nodes, citations, extensions, and lifecycle**

```

%%{ init: { 'theme': 'base', 'themeVariables': { 'primaryColor': '#1e293b', 'primaryTextColor':
'#f1f5f9', 'primaryBorderColor': '#38bdf8', 'lineColor': '#475569', 'secondaryColor': '#0f172a',
'tertiaryColor': '#0f172a', 'clusterBkg': 'rgba(30, 41, 59, 0.5)', 'clusterBorder': '#475569', 'fontSize':
'14px', 'fontFamily': 'Inter, system-ui, sans-serif', 'nodeBorder': '#38bdf8' } } }%% classDiagram
accTitle: Schema definition map for nodes, citations, extensions, and lifecycle accDescr: Organizes
the main schema object types into root, node, lifecycle, and extension clusters and shows the
conditional branches that control express-mode, CL, and SUPERSEDE_PENDING fields. namespace
Root { class RootContract { +ddr_version: string +document_profile: profile enum +active_tiers:
canonical set +nodes: DdrNode[] } } namespace NodeTypes { class DdrNode { +id: NodeId +tier:
TierEnum +status: StatusEnum +constraint_origin?: derived|imposed +prior_status?:
ACTIVE|DEPRECATED|DIRTY +parent_ids: ParentCitation[] +express_mode_group?: G1|G2|G3|G4
+extension_annotations?: namespaced map } class ParentCitation { +id: NodeId +edge_type:
derives|constrains|implements +derivation_mode?: semantic|traceability } class
ExpressProfileCondition { +document_profile = project_instance_express +requires:
express_mode_group } class CCondition { +tier = CL +allows: constraint_origin } class
SupersedePendingCondition { +status = SUPERSEDE_PENDING +requires: prior_status } }
namespace Lifecycle { class StatusTransition { +from: StatusEnum +to?: StatusEnum
+to_node_field?: prior_status +operation: OperationNameEnum +phase?: commit|rollback
+guards: GuardIdRef[] } class GuardDefinition { +id: gc-001..gc-009 +verification_mode:
structural|manual +description: string } } namespace Extensions { class ExtensionEntry { +id: string
+name: string +contract: string +reads: TierEnum[] +annotates: TierEnum[] +rules: rule[]
+scoring_profile?: profile ref } class ScoringProfile { +input_signals[] +score_bands[]
+minimum_surfacing_threshold +override_policy } } RootContract "0..*" --> DdrNode : nodes
DdrNode "0..*" --> "1" ParentCitation : parent_ids DdrNode ..> ExpressProfileCondition :
conditional DdrNode ..> CCondition : conditional DdrNode ..> SupersedePendingCondition :
conditional ExtensionEntry "0..1" ..> "1" ScoringProfile : E5 scoring_profile StatusTransition "0..*" --
> "1" GuardDefinition : guards

```

**Interpretation** The schema contract is not a flat property list; it is a graph of typed objects whose conditional relationships mirror the manual's structural, extension, and lifecycle sections.

**Authority basis** `ddr_node_schema_v6.3.yaml` `root` properties, `$defs.DdrNode`, `$defs.ParentCitation`, `$defs.ExpressModeGroup`, `$defs.ExtensionEntry`, `$defs.ScoringProfile`, `$defs.StatusTransition`, and `$defs.GuardDefinition`.

See also: Section 3, Section 5, Section 6, Section 8.

## 10. Appendices

This section collects supporting reference surfaces that remain necessary for auditability and maintenance: glossary, version history, legacy tier migration, authoritative counts, source crosswalks, practitioner quick references, and study aids. Use these appendices when you need historical context, surface counts, source-to-manual mapping, or a concise refresher on lifecycle, Express Mode, ARE, and common authoring pitfalls.

### 10.1 Glossary

| TERM                         | DEFINITION                                                                                                                                                                                                                                                                     |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Atomic Rule                  | An inclusion or exclusion constraint on a tier node, individually verifiable without reference to other nodes.                                                                                                                                                                 |
| Candidate Pool               | Extension-managed staging area for ARE-inferred nodes, explicitly outside the Core DAG until promoted via <code>INSERT</code> .                                                                                                                                                |
| DAG                          | Directed Acyclic Graph, the DDR System's foundational data structure.                                                                                                                                                                                                          |
| Dirty Flag                   | <code>DIRTY</code> status indicating a node requires re-validation following a graph-modifying event.                                                                                                                                                                          |
| Edge Type                    | One of four typed relationships: <code>derives</code> , <code>constrains</code> , <code>implements</code> , <code>extends</code> .                                                                                                                                             |
| Express Mode                 | A four-group consumption mode that can be unbundled into full-mode tiers through <code>UNBUNDLE_SCAN</code> and <code>UNBUNDLE_EXECUTE</code> .                                                                                                                                |
| Extension                    | An optional analytical overlay that reads and annotates Core nodes without modifying Core semantics.                                                                                                                                                                           |
| Leaf Node                    | A node with no children. <code>ISL</code> is the only valid leaf tier in a <code>CLEAN</code> Core DAG; non- <code>ISL</code> leaves during authoring are incomplete and are flagged by <code>VERIFY</code> .                                                                  |
| Merge Node                   | <code>SAL</code> , the point where <code>FCL</code> derivations and <code>CL</code> constraints converge.                                                                                                                                                                      |
| Orphan                       | A non-root node with no valid <code>parent_id</code> ; this is a structural violation.                                                                                                                                                                                         |
| Root Node                    | <code>XPD</code> if active, otherwise <code>SIL</code> ; the only node allowed to have an empty <code>parent_ids</code> list.                                                                                                                                                  |
| <code>REVIEW_REQUIRED</code> | A <code>VALIDATE</code> output status emitted for each semantic atomic inclusion rule. It requires a human disposition of <code>APPROVED</code> or <code>REJECTED</code> with rationale before the target node may transition from <code>DRAFT</code> to <code>ACTIVE</code> . |
| Tier Contamination           | Presence of content that violates a tier's atomic exclusion rules.                                                                                                                                                                                                             |

|                   |                                                                                                                                                                                                                            |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| verification_mode | Required field on every atomic inclusion rule. structural rules are mechanically verifiable by pattern matching, schema validation, keyword detection, or citation-graph traversal; semantic rules require human judgment. |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**HISTORICAL SCOPE** Sections 10.2 and 10.3 preserve legacy tier names, removed operations, and migration mappings strictly for version-history and migration reference. Those historical terms are not part of current-state DDR v6.3 vocabulary outside these appendices.

## 10.2 Version history

| VERSION | DATE          | SUMMARY                                                                                                                                                                                                                                         |
|---------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0     | none recorded | Initial DDR concept using a 7-tier linear model BRD -> NFR -> FSD -> SAD -> ICD -> TDD -> ISP                                                                                                                                                   |
| 2.1     | 2026-02-26    | Refined Core plus Extension system                                                                                                                                                                                                              |
| 3.0     | 2026-02-26    | Complete redesign: fork-join DAG, GPCL isolation, optional XPD, Z-axis extensions, Express Mode, CRR, and 9 Extensions                                                                                                                          |
| 3.1.1   | 2026-02-26    | Structural consolidation: universal node format, 6-edge vocabulary, and axiom implications                                                                                                                                                      |
| 4.0     | 2026-02-26    | Structural simplification: 11 -> 9 tiers, 6 -> 4 edge types, 11 -> 7 operations, fork-join to merge-node, RELOCATE removed, ARE Candidate Pool added, Express Mode reduced to 4 groups, Service Model removed, CRR removed                      |
| 5.0     | 2026-03-25    | Issue-driven refinement: SUPERSEDE_PENDING, prior_status, verification_mode, FCL-R7, ARE tri-state lifecycle, DDE-R5, UNBUNDLE_SCAN, UNBUNDLE_EXECUTE, GPCL-FCL-BR1, constraint_origin, CL-R9-imposed, manifest schema, derivation_mode, CIT-R6 |
| 6.0     | 2026-03-25    | Major version increment and versioning alignment                                                                                                                                                                                                |
| 6.1     | 2026-03-27    | Semantic-gap classification, INV-7, AX-5 wording refinement, optional semantic consistency review under VERIFY, explicit conflict resolution protocol, deferred fragments, INV-8, CIT-R7, schema alignment                                      |
| 6.2     | 2026-03-27    | Schema hardening: profile-aware root contract, typed lifecycle transitions, DELETE modeled as an operation sink, closed guard references, ParentCitation restrictions, derivation_mode gating, tier/id binding, CL-only                         |

|     |            |                                                                                                                                                                                                                                                                          |
|-----|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     |            | constraint_origin, SUPERSEDE_PENDING-only prior_status, express-node enforcement, extension shadow-key blocking                                                                                                                                                          |
| 6.3 | 2026-03-28 | Issue-resolution release: explicit document_profile, canonical active_tiers closure, status_transitions as sole lifecycle authority, deterministic ARE hardening, normalized operation namespace, centralized rule-ID typing, and closed Express Mode group requirements |

### 10.3 Tier migration

Migration policy:

- All future migrations must include a complete rule-level cross-reference table with explicit consolidation status.

Tier map:

| FROM | TO   | NOTES                                                                          |
|------|------|--------------------------------------------------------------------------------|
| XPD  | XPD  | Unchanged                                                                      |
| SIL  | SIL  | Unchanged                                                                      |
| GPCL | GPCL | Expanded to absorb ORL quality and performance content                         |
| ORL  | GPCL | ORL-R1 through ORL-R7 became GPCL-R6 through GPCL-R10, with some consolidation |
| FCL  | FCL  | Now derives from GPCL instead of ORL                                           |
| HIL  | CL   | HIL-R1 through HIL-R5 became CL-R6 through CL-R8                               |
| TDL  | CL   | TDL-R1 through TDL-R6 became CL-R1 through CL-R5                               |
| SAL  | SAL  | Simplified from fork-join to single merge node                                 |
| ICL  | ICL  | Unchanged                                                                      |
| CDL  | CDL  | Unchanged                                                                      |
| ISL  | ISL  | References CL instead of TDL for language targets                              |

Rule map:

| FROM<br>RULE | TO<br>RULE | CONSOLIDATION<br>STATUS | NOTES |
|--------------|------------|-------------------------|-------|
|--------------|------------|-------------------------|-------|

| IDS                    | IDS      |                  |                                                         |
|------------------------|----------|------------------|---------------------------------------------------------|
| ORL-R1                 | GPCL-R6  | 1:1              | Maps to quantifiable performance targets                |
| ORL-R2                 | GPCL-R7  | 1:1              | Maps to reliability and availability targets            |
| ORL-R3                 | GPCL-R8  | 1:1              | Maps to security requirements                           |
| ORL-R4                 | GPCL-R10 | 1:1              | Maps to parent <small>SIL</small> citation rule         |
| ORL-R5                 | GPCL-R9  | N:1              | Consolidated with ORL-R6                                |
| ORL-R6                 | GPCL-R9  | N:1              | Consolidated with ORL-R5                                |
| ORL-R7                 | GPCL-R9  | Absorbed         | Semantics subsumed under broader governance constraints |
| HIL-R1, HIL-R2, HIL-R3 | CL-R6    | N:1 Consolidated | Consolidated into hardware envelopes rule               |
| HIL-R4                 | CL-R7    | 1:1              | Direct map                                              |
| HIL-R5                 | CL-R8    | 1:1              | Direct map                                              |
| TDL-R1                 | CL-R1    | 1:1              | Direct map                                              |
| TDL-R2, TDL-R6         | CL-R2    | N:1 Consolidated | Consolidated into minimum version bounds                |
| TDL-R3                 | CL-R3    | 1:1              | Direct map                                              |
| TDL-R4                 | CL-R4    | 1:1              | Direct map                                              |
| TDL-R5                 | CL-R5    | 1:1              | Direct map                                              |

#### 10.4 Authoritative counts and current errata state

| SURFACE                                      | COUNT |  |
|----------------------------------------------|-------|--|
| Top-level specification sections             | 26    |  |
| Top-level schema properties                  | 26    |  |
| Document profiles                            | 3     |  |
| Canonical <code>active_tiers</code> variants | 4     |  |
| Axioms                                       | 7     |  |

|                                 |    |
|---------------------------------|----|
| Edge types                      | 4  |
| Node schema fields              | 13 |
| Citation rules                  | 7  |
| DAG invariants                  | 8  |
| Tier definitions                | 9  |
| Canonical operations            | 8  |
| Consumption modes               | 2  |
| Express groups                  | 4  |
| Extension catalog entries       | 9  |
| ARE scoring profiles            | 3  |
| Compliance checklist categories | 3  |
| Glossary entries                | 14 |
| Version history entries         | 10 |
| Representative nodes            | 9  |
| Status transitions              | 12 |
| Guard definitions               | 9  |
| Active errata entries           | 0  |

## 10.5 Source crosswalk

| SOURCE SURFACE        | MANUAL SECTION |
|-----------------------|----------------|
| project               | 2.1            |
| system_metadata       | 2.1, 2.2, 2.3  |
| errata_log            | 2.4, 10.4      |
| axioms                | 3.1            |
| node_schema_fields    | 3.5            |
| edge_type_definitions | 3.7            |
|                       |                |

|                                           |               |
|-------------------------------------------|---------------|
| dag_invariants                            | 3.9           |
| node_id_format                            | 3.6           |
| citation_rules                            | 3.8           |
| nodes                                     | 3.4, 4.1-4.9  |
| tier_definitions                          | 4.1-4.9       |
| lifecycle                                 | 5.1-5.3, 10.6 |
| operations.core_operations                | 5.4, 10.6     |
| operations.dirty_flag_triggers            | 5.5, 10.6     |
| operations.dirty_classification           | 5.5, 10.6     |
| operations.supersede_dirty_behavior       | 5.5, 10.6     |
| operations.conflict_resolution_protocol   | 5.6, 7.2      |
| operations.resolution_workflow            | 5.6           |
| operations.reconciliation_manifest_tracks | 7.3           |
| operations.reconciliation_manifest_schema | 7.3, 10.6     |
| operations.semantic_consistency_rules     | 5.4, 7.5      |
| consumption_modes                         | 6.1, 10.7     |
| express_mode                              | 6.2-6.4, 10.7 |
| constraint_precedence                     | 7.1-7.2       |
| compliance_checklist                      | 7.4-7.5       |
| extension_system                          | 8.1-8.4, 10.8 |
| extension_catalog                         | 8.5           |
| are_scoring_profiles                      | 8.4, 10.8     |
| glossary                                  | 10.1          |
| version_history                           | 10.2          |
| tier_migration                            | 10.3          |
| <b>Schema</b> properties.document_profile | 3.2, 9.1      |
| <b>Schema</b> properties.active_tiers     | 3.3, 9.2      |
| <b>Schema</b> \$defs.DdrNode              | 3.5, 9.3      |
| <b>Schema</b> \$defs.ParentCitation       | 3.8, 9.4      |



|                                                                                                                                                                        |          |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Schema \$defs.ExpressModeGroup                                                                                                                                         | 6.2, 9.5 |
| Schema \$defs.ExtensionEntry                                                                                                                                           | 8.5, 9.6 |
| Schema \$defs.ScoringProfile                                                                                                                                           | 8.4, 9.6 |
| Schema \$defs.StatusTransition                                                                                                                                         | 5.2, 9.7 |
| Schema \$defs.GuardDefinition                                                                                                                                          | 5.3, 9.7 |
| Schema enum and ID defs (StatusEnum, OperationNameEnum, TransitionPhase, TransitionSideEffect, GuardIdRef, InvariantId, CitationRuleId, AtomicRuleId, ExtensionRuleId) | 9.8      |

### 10.6 Lifecycle and Operation Quick Reference

| OPERATION | PRIMARY SCOPE                    | MUTATES GRAPH | TYPICAL MANIFEST OR REVIEW IMPACT                                                                                  | COMMON FOLLOW UP                                              |
|-----------|----------------------------------|---------------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| INSERT    | Add one node                     | Yes           | None by default; if inserted as <code>DRAFT</code> , later review and validation remain pending                    | VALIDATE<br>draft-to- <code>REVIEW_REQUIRED</code>            |
| DELETE    | Remove one node                  | Yes           | Manifest updated as orphan fallout is detected                                                                     | MODIFY<br>casca<br>DELETED<br>SUPERSEDE<br>affect child nodes |
| MODIFY    | Change one node's content        | Yes           | May lead to <code>REVIEW_REQUIRED</code> items after re-validation                                                 | VALIDATE<br>VERIFY                                            |
| SUPERSEDE | Transactionally replace one node | Yes           | <code>SUPERSEDE_FAILED</code> on failed insert or rewire;<br><code>SUPERSEDE_PENDING_DETECTED</code> if incomplete | Retry<br>rollback<br>VERIFY                                   |
| VERIFY    | Whole-graph traversal            | No            | May emit structural findings and optional semantic <code>REVIEW_REQUIRED</code> items                              | Resolve findings<br>re-run                                    |
| VALIDATE  | Single-node                      | No            | Emits violated rule IDs                                                                                            | Human review                                                  |

|                  |                      |     |                                                                                                          |                                            |
|------------------|----------------------|-----|----------------------------------------------------------------------------------------------------------|--------------------------------------------|
|                  | atomic ruleset       |     | and semantic<br>REVIEW_REQUIRED items plus<br>validation_scope                                           | dispo<br>sema<br>findir<br>befor<br>activa |
| UNBUNDLE_SCAN    | One Express<br>group | No  | May drive deferred-<br>rationale entries in the<br>manifest                                              | Anno<br>defer                              |
| UNBUNDLE_EXECUTE | One Express<br>group | Yes | Rejection payload is full<br>scan output; successful<br>execution may record<br>deferred rationale usage | VERIFI<br>result<br>mode                   |

Status and transition reminders:

- `SUPERSEDE_PENDING` is transient and blocks `CLEAN` until commit or rollback finishes.
- `DEPRECATED` is structurally present and traversed by `VERIFY`; it is not equivalent to deleted.
- `DRAFT` nodes may exist in the DAG but are excluded from `CLEAN` compliance until validated.
- `DIRTY` from `SUPERSEDE` rewiring is initially structural and scoped to immediate children.

## 10.7 Express Mode Authoring and Unbundling Reference

| GROUP | TIERS             | EXPLICIT TIER<br>TAGS<br>REQUIRED BY<br>SOURCE RULE | AUTHORING<br>IMPLICATION                                                                                        |
|-------|-------------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| G1    | XPD, SIL,<br>GPCL | Yes                                                 | Ethical, strategic, and governance<br>fragments must be explicitly<br>separable for deterministic<br>unbundling |
| G2    | FCL, CL           | Yes                                                 | Capability behavior must stay<br>separable from declared technology<br>and infrastructure constraints           |
| G3    | SAL, ICL          | Not explicitly required,<br>but still helpful       | Architectural decomposition and<br>machine contracts should stay<br>visually distinct                           |
| G4    | CDL, ISL          | Not explicitly required,<br>but still helpful       | Design blueprints and executable<br>scaffolds are easier to audit when                                          |

#### Authoring checklist:

- Use explicit `[TIER]` markers throughout `G1` and `G2`.
- Use `[DEFER]` only when you can record a human rationale in the reconciliation manifest.
- Treat `UNBUNDLE_SCAN` as a workflow tool, not as a one-time gate.
- Expect `UNBUNDLE_EXECUTE` to reject atomically if any undeferred fragment remains `ambiguous` or `none`.

#### Illustrative grouped content:

[FCL] User can export monthly billing reports.

[CL] Export worker must run in Python 3.10+ on Linux.

[DEFER] Accessibility wording pending UX review.

### 10.8 ARE and Extension Reference

| SURFACE                     | SOURCE-DERIVED RULE OF THUMB                                                                                                                                          | WHY IT MATTERS                                                                          |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Extension mutation boundary | Extensions read, annotate, advise, and generate artifacts; they do not mutate Core node content, <code>parent_ids</code> , <code>tier</code> , or <code>status</code> | Preserves AX-6 declarative integrity and keeps Core authority local to named operations |
| Annotation storage          | Extension metadata lives in <code>extension_annotations</code> only, with namespaced keys                                                                             | Prevents <code>extends</code> from masquerading as Core lineage                         |
| Candidate Pool              | Only E5 uses a candidate pool, and it stays outside the Core DAG                                                                                                      | ARE inference remains reviewable without silently altering authoritative structure      |
| Paused ARE state            | Pool must be checkpointed on entry and after every mutating pool action while paused                                                                                  | Restart behavior is part of the contract, not optional implementation polish            |
| Disabled ARE state          | Pool is discarded, checkpoint deleted, visibility removed, and promotion/discard actions disabled                                                                     | Avoids stale candidate carryover after shutdown                                         |
| Review threshold            | Candidates below threshold need <code>override_flag: true</code> plus non-empty <code>human_rationale</code>                                                          | Prevents low-confidence inference from entering review without human accountability     |
| Concrete                    | <code>standard_v1</code> and <code>conservative_v1</code> are                                                                                                         | Prevents teams from treating                                                            |

|          |                                                               |                                                                    |
|----------|---------------------------------------------------------------|--------------------------------------------------------------------|
| profiles | reusable profiles; <code>custom</code> is a template contract | the <code>custom</code> template as a pre-approved scoring profile |
|----------|---------------------------------------------------------------|--------------------------------------------------------------------|

## 10.9 Section Summaries

| SECTION | CORE TAKEAWAY                                                                                       | COMMON FAILURE MODE                                                                                     | REVISIT WHEN...                                                            |
|---------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| 1       | The YAML pair is authoritative; this manual is derivative guidance                                  | Treating prose summaries as stronger than source YAML                                                   | You need to settle a disagreement between manual text and machine contract |
| 2       | v6.3 is framed by explicit profiling, lifecycle closure, ARE hardening, and operation normalization | Reading v6.3 as a minor editorial revision instead of a closure release                                 | You need to understand why later sections got stricter                     |
| 3       | The Core model is a typed DAG with closed citation and profile rules                                | Mixing <code>extends</code> into Core lineage or skipping tiers                                         | You are authoring topology, IDs, citations, or profile roots               |
| 4       | Each tier has a sharply bounded semantic role and rule surface                                      | Writing implementation detail too early or policy content too low                                       | You are deciding where content belongs                                     |
| 5       | Lifecycle behavior is a closed machine and operations are named, typed entry points                 | Treating <code>SUPERSEDE</code> as a rename or assuming all <code>DIRTY</code> cascades behave the same | You are modifying or replacing nodes                                       |
| 6       | Express Mode is grouped Full Mode, not a reduced variant                                            | Assuming ambiguous fragments can be auto-placed during execute                                          | You are authoring or unbundling grouped documents                          |
| 7       | CLEAN depends on status, topology, rule                                                             | Treating CLEAN as equivalent to “no                                                                     | You are asserting readiness or                                             |

|    |                                                                                      |                                                                                    |                                                                       |
|----|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
|    | conformance, manifest state, and advisory disposition                                | DIRTY nodes" only                                                                  | validation completeness                                               |
| 8  | Extensions are tightly bounded overlays; ARE is special because it stages candidates | Letting extension output blur into Core authority                                  | You are integrating analytical tooling or reviewing ARE output        |
| 9  | The schema is an active contract with conditionals, closed enums, and typed IDs      | Reading the schema as passive documentation instead of executable validation logic | You are implementing validators or authoring machine-facing artifacts |
| 10 | The appendices are operational tools, not just historical leftovers                  | Skipping quick references and recreating rules from memory                         | You need fast lookup, study aids, or migration context                |

## 10.10 Pro Tips

- Use `derivation_mode: traceability` only when the parent is lineage authority rather than source semantics.
- Treat `SAL` as the only merge node. If you find yourself inventing another one, the model is drifting.
- Keep `FCL` behavioral and `CL` declarative. Mixed nodes look efficient at first and create downstream ambiguity later.
- When `constraint_origin = imposed`, record the external authority source immediately. Retroactive cleanup is expensive.
- Use `UNBUNDLE_SCAN` early in `G1` and `G2` authoring. Late ambiguity is harder to unwind.
- Do not treat `DEPRECATED` as inactive. It still participates in graph traversal and audit history.
- Remember that `SUPERSEDE_PENDING` is blocking. A half-finished supersede is not a tolerable intermediate resting state.
- Keep extension annotations obviously namespaced and never mirror Core field names after `::`.
- If an ARE candidate is below threshold, force yourself to write the human rationale before allowing review.
- Use the reconciliation manifest as structured evidence, not as a miscellaneous note sink.
- Re-run `VERIFY` after any operation that rewires lineage, not just after obvious content edits.
- When in doubt about validity, check the schema conditionals in Section 9 before inventing a workflow exception.

## 10.11 Q&A

| QUESTION                                                                             | ANSWER                                                                                                                |
|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Is Express Mode a simplified DDR model?                                              | No. It is grouped presentation of the same Full Mode structure, with deterministic unbundling rules.                  |
| Can <code>extends</code> appear in <code>parent_ids</code> ?                         | No. <code>extends</code> belongs only in extension annotations and never in Core parent citations.                    |
| Can <code>SIL</code> be a root node when <code>XPD</code> is active?                 | No. If <code>active_tiers</code> contains <code>XPD</code> , <code>SIL</code> must have at least one parent citation. |
| Is <code>SUPERSEDE_PENDING</code> a stable lifecycle destination?                    | No. It is a transient transactional state that must commit or roll back.                                              |
| Does <code>DEPRECATED</code> remove a node from verification scope?                  | No. <code>DEPRECATED</code> nodes remain structurally present and are traversed by <code>VERIFY</code> .              |
| Can ARE promote a candidate directly into the Core DAG?                              | No. Promotion always goes through <code>INSERT</code> with full validation.                                           |
| Is the <code>custom</code> ARE profile usable as-is?                                 | No. It is a template and validation contract for implementation-defined custom profiles.                              |
| What happens if <code>UNBUNDLE_SCAN</code> returns <code>none</code> for a fragment? | <code>UNBUNDLE_EXECUTE</code> must reject unless that fragment is explicitly deferred with recorded rationale.        |
| Do structural <code>DIRTY</code> flags always cascade to grandchildren?              | No. <code>SUPERSEDE</code> rewiring creates a scoped structural <code>DIRTY</code> on immediate children only.        |
| Can extensions set Core nodes to <code>DIRTY</code> ?                                | No. Extensions may advise, but they do not mutate Core status.                                                        |
| What is the only allowed semantic-gap classification in v6.3?                        | <code>MISSING_MEDIATOR</code> .                                                                                       |
| Which document profile is required when <code>project.mode = express</code> ?        | <code>project_instance_express</code> .                                                                               |

## 10.12 Quiz

1. Which DDR surface is the sole normative authority for lifecycle transitions in v6.3?
2. Which edge type is valid in `extension_annotations` but not in `parent_ids`?
3. What field records the source node's prior lifecycle value during `SUPERSEDE`?
4. Which two Express groups require explicit tier annotations for deterministic unbundling?
5. What is the only allowed semantic-gap classification in the reconciliation manifest?
6. What ARE transition is explicitly forbidden?
7. Can a `DEPRECATED` node still be traversed by `VERIFY`?

8. What additional requirement applies to E5 in the schema that does not apply to every other extension entry?
9. What happens to the ARE checkpoint file on any transition to `disabled` ?
10. Which tier is the only valid merge node in the Core DAG?

Answer key:

1. `lifecycle.status_transitions`
2. `extends`
3. `prior_status`
4. `G1 and G2`
5. `MISSING_MEDIATOR`
6. `disabled -> paused`
7. Yes
8. `scoring_profile` is required
9. It is deleted atomically as the pool is discarded
10. `SAL`